



TRABALLO FIN DE GRAO
GRAO EN ENXEÑARÍA INFORMÁTICA
MENCIÓN EN TECNOLOXÍAS DA INFORMACIÓN



Diseño y Desarrollo de un Datamart Analítico para la Prevención del Abandono y Fracaso Escolar

Estudiante: Andrés Paz Paredes
Dirección: José Ramón Paramá Gabía

A Coruña, junio de 2026.

Dedicatoria

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Agradecimientos

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Resumen

Este Trabajo de Fin de Grado presenta el diseño y desarrollo de un Datamart Analítico para la prevención del abandono escolar. El sistema integra datos académicos y sociodemográficos desde sistemas transaccionales, utilizando Apache Hop (ETL) para procesarlos hacia un Data Warehouse en PostgreSQL. Todo este ecosistema tecnológico se ha orquestado mediante contenedores Docker, garantizando su portabilidad y escalabilidad. Durante el proceso, se calculan métricas de riesgo predictivo para identificar alumnos vulnerables. La solución final expone esta información mediante una API en Python (FastAPI) y la visualiza en un Dashboard interactivo en React, proporcionando a los centros educativos una herramienta eficaz para la toma de decisiones basada en datos.

Abstract

This Bachelor's Thesis presents the design and development of an Analytical Datamart for the prevention of school dropout. The system integrates academic and socio-demographic data from transactional systems, using Apache Hop (ETL) to process them into a Data Warehouse on PostgreSQL. This entire technological ecosystem has been orchestrated using Docker containers, ensuring its portability and scalability. During the process, predictive risk metrics are calculated to identify vulnerable students. The final solution exposes this information through a Python API (FastAPI) and visualizes it in an interactive React Dashboard, providing educational centers with an effective tool for data-driven decision-making.

Palabras clave:

- Inteligencia de Negocios
- Prevención Abandono Escolar
- Datamart
- ETL
- Apache Hop
- Contenedores Docker

Keywords:

- Business Intelligence
- School Dropout Prevention
- Datamart
- ETL
- Apache Hop
- Docker Containers

Índice general

1	Introducción	1
1.1	Contexto	1
1.2	Motivación	2
1.3	Objetivos	2
2	Estado del arte y marco teórico	4
2.1	Inteligencia de negocios y data warehousing	4
2.1.1	Metodología de Ralph Kimball	4
2.1.2	Visualización de datos y cuadros de mando	6
2.2	Procesos ETL y Apache Hop	6
2.2.1	Elección de Apache Hop	6
2.3	Arquitecturas web: cliente-servidor y APIs REST	6
2.4	Evolución de las arquitecturas: contenedores	7
3	Descripción de la aplicación web	8
3.1	Flujo de acceso y autenticación	8
3.2	Estructura general de la interfaz	9
3.2.1	Barra lateral (Sidebar)	9
3.3	Vista Macro: Centro de Mando Autonomico	9
3.3.1	Indicadores de cabecera	10
3.3.2	Panel de alertas institucionales	10
3.3.3	Mapa geográfico de centros	10
3.3.4	Gráficos de análisis macro	10
3.3.5	Ranking de centros	11
3.4	Vista Micro: Análisis de Centro	11
3.4.1	Indicadores de alumnado con comparativa interanual	11
3.4.2	Gráficos de análisis de alumnado	11
3.4.3	Directorio de alumnos	12

3.5	Gestión de usuarios	12
3.6	Casos de uso principales	13
3.6.1	Caso de uso 1: Priorización de centros para asignación de recursos de apoyo	13
3.6.2	Caso de uso 2: Diagnóstico de la brecha digital en centros de zona rural	13
3.6.3	Caso de uso 3: Evaluación del impacto de una intervención previa	13
3.6.4	Caso de uso 4: Análisis comparativo por tipo de centro	14
4	Metodología y planificación	15
4.1	Metodología de desarrollo	15
4.2	Gestión y control de versiones	16
4.3	Planificación temporal	16
4.4	Presupuesto y costes	17
4.4.1	Recursos humanos	17
4.4.2	Recursos materiales e infraestructura	18
5	Análisis y arquitectura	19
5.1	Requisitos del sistema	19
5.2	Análisis del sistema transaccional origen	20
5.3	Arquitectura tecnológica basada en Docker	20
5.3.1	Justificación del paradigma de contenedores	23
6	Diseño del datamart analítico	25
6.1	Modelo dimensional en estrella	25
6.1.1	Tablas de dimensiones en detalle	25
6.1.2	Tablas de hechos	27
6.2	Reglas de negocio y cálculo de riesgos	27
6.2.1	Riesgo socioeconómico	27
6.2.2	Riesgo de abandono del alumno	28
7	Capa ETL con Apache Hop	30
7.1	Configuración del entorno y conexiones	30
7.2	Análisis de pipelines	30
7.2.1	Volcado a staging	31
7.2.2	Dimensión de demografía familiar	31
7.2.3	Dimensión de estudiante	33
7.2.4	Dimensión de tiempo	35
7.2.5	Dimensión de asignatura	36
7.2.6	Dimensión de curso	37

7.2.7	Dimensión de adaptación	39
7.2.8	Dimensión de centro	40
7.2.9	Tabla de hechos de calificaciones	41
7.2.10	Tabla de hechos de rendimiento por evaluación	42
7.3	Orquestación y workflows	46
7.3.1	Propagación de parámetros dinámicos	47
7.3.2	Manejo de errores multinivel (<i>error handling</i>)	51
8	Desarrollo del dashboard analítico	54
8.1	Backend RESTful (FastAPI)	54
8.1.1	Arranque y ciclo de vida del contenedor	55
8.1.2	Configuración centralizada (Pydantic Settings)	55
8.1.3	Persistencia dual: esquemas app y dwh	56
8.1.4	Motor de base de datos y pool de conexiones	56
8.1.5	Migraciones con Alembic	57
8.1.6	Autenticación y autorización	57
8.1.7	Arquitectura en capas del backend	58
8.1.8	Diagrama de clases del backend	58
8.1.9	Motor de consultas analíticas (<i>crud_dashboard.py</i>)	59
8.1.10	Catálogo de endpoints	62
8.2	Frontend (React y Vite)	64
8.2.1	Pipeline de construcción y despliegue	64
8.2.2	Dependencias del proyecto	64
8.2.3	Arquitectura del código fuente	65
8.2.4	Enrutamiento y protección de rutas	65
8.2.5	Sistema de diseño (CSS nativo)	67
8.2.6	Integración con la API	67
8.3	Proxy inverso y despliegue (Nginx)	68
8.3.1	Terminación TLS y redirección HTTP	68
8.3.2	Reglas de enrutamiento	68
8.3.3	Optimizaciones de rendimiento	69
8.3.4	Topología de red resultante	69
9	Conclusiones	70
9.1	Cumplimiento de objetivos	70
9.2	Lecciones aprendidas	71
9.3	Líneas de trabajo futuro	72

A Material adicional	74
Bibliografía	76

Índice de figuras

2.1	Esquema conceptual de un modelo en estrella.	5
2.2	Interfaz de usuario de Apache Hop.	7
4.1	Diagrama de Gantt de la planificación del proyecto.	17
5.1	Modelo Entidad-Relación (ER) de la base de datos transaccional origen.	21
5.2	Esquema Relacional de origen.	22
6.1	Esquema del Datamart analítico (<i>dwh</i>) en modelo estrella: dos tablas de hechos de distinta granularidad (<i>fact_rendimiento_anual</i> y <i>fact_calificaciones</i>) rodeadas por siete dimensiones. Se indica con [SCD2] la dimensión <i>dim_estudiante</i> , que implementa una Dimensión de Variación Lenta de Tipo 2.	26
7.1	Configuración de la conexión <i>postgres-dwh</i>	31
7.2	Flujo del pipeline <i>extract_all_to_staging.hpl</i>	32
7.3	Flujo del pipeline <i>dim_demografia_familiar.hpl</i>	34
7.4	Flujo del pipeline <i>dim_estudiante.hpl</i>	35
7.5	Flujo del pipeline <i>dim_tiempo.hpl</i>	37
7.6	Flujo del pipeline <i>dim_asignatura.hpl</i>	38
7.7	Flujo del pipeline <i>dim_curso.hpl</i>	39
7.8	Flujo del pipeline <i>dim_adaptacion.hpl</i>	40
7.9	Flujo del pipeline <i>dim_centro.hpl</i>	41
7.10	Flujo del pipeline <i>fact_calificaciones.hpl</i> (Parte 1).	43
7.11	Flujo del pipeline <i>fact_calificaciones.hpl</i> (Parte 2).	44
7.12	Flujo del pipeline <i>fact_rendimiento_anual_full.hpl</i> (Parte 1).	47
7.13	Flujo del pipeline <i>fact_rendimiento_anual_full.hpl</i> (Parte 2).	48
7.14	Flujo completo del pipeline <i>fact_rendimiento_anual_incremental.hpl</i>	49
7.15	Nodo <i>Database Join</i> en <i>fact_rendimiento_anual_incremental.hpl</i>	50

7.16	Workflow principal (<i>main.hwf</i>) y su delegación jerárquica en sub-flujos. . . .	52
7.17	Sub-flujo de dimensiones (<i>dimensions.hwf</i>) ejecutado en paralelo.	52
7.18	Sub-flujo de hechos (<i>facts.hwf</i>) ejecutado de forma secuencial.	53
8.1	Diagrama de despliegue y enrutamiento del proxy inverso Nginx: separación de tráfico estático (React) y tráfico de API (FastAPI) dentro de la red Docker <i>tfg-network</i>	55
8.2	Diagrama de secuencia del flujo de autenticación JWT: fase de login (generación y almacenamiento del token) y fase de petición protegida (inyección del token por el interceptor Axios y validación por <code>get_current_user</code>).	57
8.3	Diagrama de clases simplificado del backend FastAPI: interfaces REST, lógica de negocio y acceso a datos.	60
8.4	Modelo en estrella: JOINS entre <code>fact_rendimiento_anual</code> y las seis dimensiones del datamart utilizados como base de todas las consultas analíticas del backend.	61
8.5	Catálogo de endpoints de la API del dashboard (<code>/api/v1</code>).	63
8.6	Pipeline de build y despliegue del frontend: desde el código fuente JSX hasta el servicio de ficheros estáticos a través del proxy Nginx.	64
8.7	Estructura de directorios del frontend (<code>src/</code>): separación en <code>pages/</code> (vistas), <code>components/</code> (UI reutilizable), <code>context/</code> (estado global) y <code>services/</code> (capa de datos).	66

Índice de tablas

4.1	Estimación de presupuesto de recursos humanos.	17
4.2	Estimación de costes materiales y de infraestructura (1 año).	18

Introducción

EL abandono temprano de la educación tiene importantes implicaciones sociales en términos de calidad de vida, salud, marginalidad social, delincuencia o empleabilidad. Dado el importante coste socioeconómico que este fenómeno conlleva a largo plazo, todas las administraciones públicas y organismos gubernamentales tratan de reducir en lo posible la tasa de abandono temprano y fracaso escolar mediante políticas de intervención y apoyo proactivo [1].

1.1 Contexto

La administración pública, habitualmente a nivel de comunidad autónoma, es la entidad encargada de la gestión de la educación y debe enfrentarse al complejo desafío de realizar un reparto equitativo de los escasos recursos materiales y personales entre su extensa red de centros educativos.

La casuística de cada institución es única, y algunos centros necesitan un volumen de apoyo significativamente mayor que otros debido a múltiples factores sistémicos y estructurales. Estar ubicados en zonas económicamente deprimidas, contar con un alto porcentaje de alumnado proveniente de clases desfavorecidas o enfrentarse a severas barreras de brecha digital son elementos que lastran el rendimiento global de un centro. Esto se alinea con la realidad del indicador AROPE en España, que evidencia cómo un porcentaje significativo de las familias conviven en situaciones de vulnerabilidad o pobreza relativa [2, 3]. Abordar estas carencias desde el prisma de la "pobreza multidimensional" [4, 5] es fundamental para que la administración actúe como un equilibrador social efectivo.

Sin embargo, históricamente la información ha estado dispersa en bases de datos heterogéneas a nivel gubernamental. Para hacer un reparto verdaderamente objetivo y eficiente, la administración no puede depender de intuiciones o reportes fragmentados; debe disponer de una "fotografía" exacta del estado de cada centro, construida sobre medidas estandarizadas,

comparables y objetivas a nivel autonómico.

1.2 Motivación

Ante este escenario de gran volumen y dispersión de datos, surge la necesidad imperativa de aplicar técnicas de Inteligencia de Negocios (*Business Intelligence* o BI) a la gestión pública. Dado el inmenso número de centros educativos que suelen estar bajo la tutela de las administraciones, la toma de decisiones manual y aislada resulta inabarcable.

La motivación fundamental de este Trabajo de Fin de Grado es el diseño y desarrollo de un ecosistema analítico que culmine en la creación de un cuadro de mando. Dicho cuadro de mando estará compuesto por una serie de Indicadores Clave de Desempeño (KPIs, por sus siglas en inglés) que sinteticen y crucen de forma inteligente el rendimiento académico con el contexto sociodemográfico de los centros a escala global.

El propósito final es que la tecnología se ponga al servicio de los gestores públicos, sirviendo como una herramienta centralizada y accionable para guiar la difícil tarea de asignación de recursos, distribución de presupuestos y dotación de personal de apoyo entre los distintos centros educativos de forma totalmente empírica y justificada.

1.3 Objetivos

El objetivo principal de este proyecto es crear un almacén de datos (Data Warehouse) que extraiga su información a partir de diversas fuentes institucionales, y el desarrollo de un cuadro de mando que muestre distintos KPIs de interés estratégico para los gestores públicos.

Para alcanzar este propósito, garantizando además que todo el sistema se diseñe utilizando exclusivamente herramientas libres de licencias de pago, el trabajo se ha desglosado en los siguientes objetivos específicos:

1. **Simulación de fuentes de datos:** Crear tres bases de datos de origen que representen el entorno autonómico: una con expedientes académicos de alumnos, una con adaptaciones curriculares y una tercera con los resultados de encuestas socioeconómicas familiares.
2. **Diseño del almacén de datos:** Modelar un repositorio centralizado bajo un enfoque dimensional (Datamart) que aglutine e integre la información de las tres fuentes mencionadas para ser el origen analítico de los KPIs.
3. **Diseño de procesos ETL:** Diseñar e implementar los flujos de Extracción, Transformación y Carga (ETL) que pueblen los datos de las fuentes de origen hacia el almacén de datos.

4. **Desarrollo de la lógica de negocio (Backend):** Desplegar una API robusta y segura que sirva como capa de servicio estandarizada entre el almacén de datos y la interfaz gráfica.
5. **Creación de la aplicación web (Dashboard):** Implementar el cuadro de mando interactivo destinado a los gestores, diseñado para mostrar y explorar visualmente los KPIs desglosados por centro, localidad y período de tiempo.

De este modo, el proyecto abarca de manera secuencial e iterativa la totalidad del ciclo de vida de un proyecto de Inteligencia de Negocios aplicado a la administración pública.

Estado del arte y marco teórico

PARA comprender adecuadamente las decisiones arquitectónicas y técnicas tomadas en este proyecto, es fundamental contextualizar los conceptos teóricos subyacentes. Este capítulo describe los fundamentos de la Inteligencia de Negocios y las herramientas ETL que sustentan el Datamart desarrollado.

2.1 Inteligencia de negocios y data warehousing

La Inteligencia de Negocios (*Business Intelligence* o BI) comprende el conjunto de metodologías, aplicaciones y tecnologías que permiten reunir, depurar y transformar datos de sistemas transaccionales en información estructurada, para su explotación directa o para su análisis y conversión en conocimiento.

En el ámbito de la gestión educativa pública, la aplicación de la Inteligencia de Negocios trasciende el mero análisis de expedientes individuales para convertirse en una herramienta estratégica de gobernanza. La explotación masiva de datos a nivel autonómico o gubernamental permite a las administraciones abandonar los modelos de reparto de recursos basados en la inercia institucional o informes fragmentados. Al aplicar técnicas de BI sobre bases de datos demográficas, económicas y académicas consolidadas, los gestores públicos pueden identificar patrones de vulnerabilidad sistémica, comparar objetivamente las necesidades entre distintos centros y diseñar políticas educativas proactivas fundamentadas en evidencias empíricas (*Data-Driven Policy Making*).

2.1.1 Metodología de Ralph Kimball

Para el diseño del repositorio de datos se ha optado por la metodología ascendente o *Bottom-Up* propuesta por Ralph Kimball [6]. A diferencia de enfoques transaccionales clásicos, Kimball propone construir el almacén de datos (Data Warehouse) a partir de la integración de diferentes *Datamarts* temáticos.

Un Datamart es un subconjunto orientado a una área específica (en este caso, la orientación y prevención del abandono escolar). La estructura central de esta metodología es el modelo dimensional, habitualmente implementado mediante esquemas en estrella (ver Figura 2.1). Para comprender este diseño, es esencial entender dos conceptos arquitectónicos clave:

- **Modelo en Estrella:** Consiste en separar radicalmente la información en dos tipos de tablas. En el centro de la "estrella" se sitúan las *Tablas de Hechos* (*Fact Tables*), que almacenan eventos medibles y cuantitativos (por ejemplo, las calificaciones o el riesgo de abandono). Alrededor, conectadas directamente a la tabla de hechos, orbitan las *Tablas de Dimensiones* (*Dimension Tables*), que contienen el contexto descriptivo de esos eventos (el "quién", "cuándo", "dónde" y "qué": estudiante, tiempo, centro, curso).
- **Desnormalización Intencionada:** Mientras que las bases de datos operativas tradicionales (OLTP) aplican la "normalización" (dividir los datos en múltiples tablas para evitar redundancias y proteger la escritura), los Datamarts (OLAP) hacen exactamente lo contrario. Se aplica la *desnormalización*, es decir, se admite la redundancia de datos (aplanando jerarquías como "Ciclo → Especialidad → Curso" en una sola tabla) con el objetivo de reducir drásticamente las operaciones matemáticas de cruce (JOINS). Esto permite que las consultas analíticas sobre grandes volúmenes de datos sean extremadamente rápidas para alimentar cuadros de mando.



Figura 2.1: Esquema conceptual de un modelo en estrella.

2.1.2 Visualización de datos y cuadros de mando

El objetivo final de cualquier arquitectura de Inteligencia de Negocios es la correcta comunicación de los hallazgos a los responsables de la toma de decisiones. Teóricamente, un *Dashboard* o Cuadro de Mando Integral es una interfaz visual que proporciona un resumen centralizado de los Indicadores Clave de Rendimiento (KPIs). El estado del arte en el diseño de estas interfaces aboga por principios de "carga cognitiva reducida", priorizando la legibilidad, el uso de jerarquías de información y el empleo estratégico del color para señalar desviaciones críticas. Además, es fundamental la capacidad de *drill-down* (profundización analítica), que permite al gestor público navegar de forma fluida desde una visión macroscópica a nivel de toda la comunidad o municipio, hasta diseccionar el estado de vulnerabilidad de un centro educativo específico.

2.2 Procesos ETL y Apache Hop

Los procesos ETL (Extracción, Transformación y Carga) son el motor de cualquier sistema BI. Son los encargados de extraer los datos de las fuentes originales, aplicarles reglas de limpieza, filtrado y transformación, y finalmente cargarlos en el Datamart dimensional.

2.2.1 Elección de Apache Hop

Apache Hop (*Hop Orchestration Platform*) [7] es una plataforma moderna de ingeniería de datos y orquestación. Nace como una bifurcación (*fork*) de Pentaho Data Integration (PDI), con el objetivo de modernizar su arquitectura, mejorar su integración con entornos de contenedores y facilitar su uso a través de una interfaz de usuario web (*Hop Web*).

La elección de Apache Hop frente a otras herramientas del mercado se fundamenta en su curva de aprendizaje, su modelo visual *drag-and-drop* (ver Figura 2.2) y, especialmente, su excelente soporte para la ejecución nativa en contenedores Docker, lo cual encaja perfectamente con la arquitectura diseñada para este proyecto.

2.3 Arquitecturas web: cliente-servidor y APIs REST

El diseño de aplicaciones analíticas modernas ha abandonado los modelos donde el servidor generaba directamente las vistas que el usuario consumía. El marco teórico actual se fundamenta en el paradigma **Cliente-Servidor desacoplado**. En este modelo, el *Frontend* (interfaz de usuario) y el *Backend* (lógica de negocio y base de datos) operan como aplicaciones completamente independientes.

Para lograr la comunicación entre ambas capas, el estándar universal son las **APIs RESTful** (*Representational State Transfer*). Una API REST es una interfaz que utiliza el protocolo

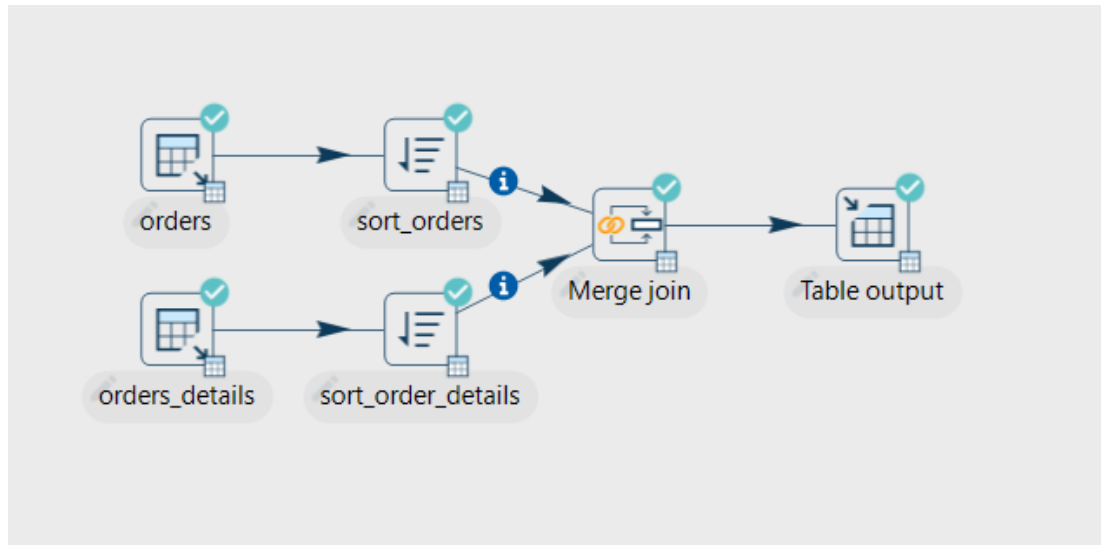


Figura 2.2: Interfaz de usuario de Apache Hop.

HTTP para solicitar y enviar datos. Bajo este enfoque, el servidor expone *endpoints* a los que el cliente realiza peticiones para obtener la información analítica. Las respuestas se estructuran habitualmente en formato JSON (*JavaScript Object Notation*), un estándar ligero que permite al *Frontend* procesar ágilmente los datos y renderizarlos en gráficos interactivos. Este desacoplamiento es esencial para asegurar la escalabilidad del sistema.

2.4 Evolución de las arquitecturas: contenedores

El estado del arte en el despliegue de infraestructuras de datos ha sufrido un cambio de paradigma radical en los últimos años. Tradicionalmente, proyectos complejos que involucraban múltiples bases de datos, flujos ETL y servidores web se desplegaban como arquitecturas monolíticas o empleaban máquinas virtuales que consumían ingentes recursos del sistema operativo anfitrión.

La introducción de la **contenedorización**, abanderada por plataformas como Docker, representa el estándar actual de la industria. Teóricamente, un contenedor encapsula un microservicio individual junto con todas sus librerías y archivos de configuración, aislándolo completamente. Esta filosofía de "aislamiento de procesos ligeros" garantiza la inmutabilidad del software (eliminando el clásico problema de diferencias entre entornos de desarrollo y producción). Para un ecosistema educativo, donde la infraestructura tecnológica de los colegios puede ser muy limitada y heterogénea, apoyarse en este marco teórico es indispensable para asegurar un despliegue reproducible, ágil e independiente del hardware subyacente.

Descripción de la aplicación web

Los capítulos precedentes han establecido el marco teórico que sustenta este proyecto: los fundamentos del modelado dimensional, los procesos ETL y las arquitecturas de visualización analítica.

El resultado del proyecto es un cuadro de mando analítico accesible desde el navegador, cuyo usuario objetivo es el **técnico o inspector de la administración educativa autonómica** responsable de la distribución de recursos entre centros. La herramienta le permite responder a dos preguntas fundamentales de forma secuencial: en primer lugar, *qué centros requieren mayor atención o dotación de recursos* en el conjunto de la comunidad autónoma; y en segundo lugar, *por qué* un centro concreto presenta el índice de riesgo que presenta, desagregando las causas académicas, socioeconómicas y curriculares que lo explican.

Para dar respuesta a estas dos preguntas, la interfaz se estructura en dos vistas complementarias que comparten los mismos indicadores de alerta pero los miden a distinta escala. La **vista Macro** opera a escala autonómica: sus KPI cuentan el número de *centros* que superan la mediana autonómica en cada métrica de riesgo. La **vista Micro** opera a escala de centro: sus KPI cuentan el número de *alumnos* de los centros seleccionados que presentan cada factor de riesgo. El flujo natural de uso consiste en identificar centros prioritarios en la vista Macro y, a continuación, realizar un *drilldown* hacia la vista Micro para diagnosticar las causas concretas del riesgo en esos centros, fundamentando así la decisión de asignación de recursos con evidencia objetiva.

3.1 Flujo de acceso y autenticación

La **pantalla de login** solicita únicamente las credenciales de acceso (correo electrónico y contraseña). Tras una autenticación correcta, la aplicación almacena el token de sesión de forma segura y redirige automáticamente al usuario a la vista principal del cuadro de mando. La sesión persiste entre visitas: si el usuario cierra el navegador y lo vuelve a abrir, la aplicación

la recupera automáticamente sin necesidad de volver a introducir las credenciales, siempre que el token no haya expirado.

3.2 Estructura general de la interfaz

Tras el login, el usuario accede a un *layout* de aplicación compuesto por dos áreas diferenciadas: una barra lateral de navegación a la izquierda y el área de contenido principal a la derecha.

3.2.1 Barra lateral (Sidebar)

La barra lateral cumple una doble función: sirve como menú de navegación entre las vistas de la aplicación y alberga los **filtros globales** que condicionan todos los datos mostrados en pantalla de forma simultánea. Puede contraerse para maximizar el espacio de visualización, mostrando únicamente los iconos de navegación.

Los filtros globales disponibles son cuatro. El **curso académico** permite seleccionar el año lectivo que se desea analizar; por defecto, la aplicación selecciona automáticamente el curso más reciente disponible en el sistema. El **tipo de centro** filtra por tipología institucional (IES, CEIP, etc.). El **centro educativo** permite seleccionar uno o varios centros concretos mediante un selector con búsqueda por nombre, lo que habilita el drilldown desde la vista Macro hacia la Micro. El **ciclo formativo** restringe el análisis a un ciclo educativo específico. Cualquier cambio en estos filtros se propaga de forma inmediata a todas las visualizaciones activas, sin necesidad de confirmar ni recargar la página. Un botón de *Resetear* restaura los filtros a su estado por defecto.

La barra lateral también incluye un selector de **tema visual** (modo claro / modo oscuro), que se persiste entre sesiones mediante *localStorage*, y el botón de cierre de sesión. En cuanto a la navegación, el menú ofrece acceso a tres secciones: **Centro de Mando** (*/macro*), **Análisis de Centro** (*/micro*), y **Gestión de Usuarios** (*/users*), esta última visible únicamente para cuentas con rol de administrador.

3.3 Vista Macro: Centro de Mando Autonómico

La vista *Centro de Mando Autonómico* es el punto de partida habitual del flujo de trabajo. Ofrece una perspectiva agregada del conjunto de centros educativos de la comunidad autónoma, permitiendo al técnico de la administración identificar qué centros concentran mayor riesgo y, por tanto, deben ser priorizados en la asignación de recursos de apoyo, refuerzo o inspección.

3.3.1 Indicadores de cabecera

En la parte superior de la vista se presentan tres **tarjetas KPI** de resumen que ofrecen una lectura inmediata del estado del sistema: el número total de centros incluidos en la selección activa, el número de centros con índice de riesgo institucional igual o superior a 7 sobre 10 (centros críticos), y la media autonómica del índice de riesgo de todos los centros analizados.

3.3.2 Panel de alertas institucionales

Debajo de los KPI de cabecera, un panel de **alertas institucionales** desglosa diez métricas de riesgo a escala de centro. Para cada métrica, el indicador muestra el número de centros que superan la mediana autonómica del número de alumnos afectados, calculada a partir de los datos individuales de la vista Micro. Las métricas cubren: alto riesgo de abandono, alumnado con suspensos, desfase académico por edad, repetidores, riesgo socioeconómico alto, brecha digital, bajo nivel de estudios familiar, adaptaciones curriculares, repetidores en 1.º y 2.º de Primaria, y suspensos en 1.º y 2.º de Primaria.

Estas tarjetas son interactivas: al pulsar sobre cualquiera de ellas, la vista filtra automáticamente el ranking de centros para mostrar únicamente aquellos que superan la mediana en esa métrica concreta, desplazándose de forma suave hasta la sección del ranking. Este mecanismo permite al técnico focalizar la atención en el subconjunto de centros más afectados por un factor de riesgo específico antes de decidir qué tipo de recurso asignar.

3.3.3 Mapa geográfico de centros

Un **mapa interactivo** muestra la distribución geográfica de todos los centros. Cada centro se representa mediante un marcador circular cuyo color refleja su nivel de riesgo (verde, amarillo o rojo). Al hacer clic sobre un marcador, la aplicación realiza un drilldown automático: fija ese centro en los filtros globales y navega a la vista Micro para mostrar el diagnóstico causal de su índice de riesgo.

3.3.4 Gráficos de análisis macro

La vista incluye cuatro gráficos complementarios. La **evolución del riesgo autonómico** es una línea temporal con la media histórica del índice de riesgo de los centros, que permite identificar si la tendencia general mejora o empeora a lo largo de los cursos académicos. La **comparativa por tipo de centro** es un gráfico de barras que contrasta la media del índice de riesgo según la tipología institucional, revelando si determinados tipos de centro concentran sistemáticamente mayor vulnerabilidad. La **distribución del riesgo institucional** agrupa los centros en tres niveles (Bajo 0–3, Medio 4–6, Alto 7–10) mediante barras horizontales; al hacer clic sobre una barra, el ranking inferior se filtra para mostrar únicamente los centros

de ese nivel. Por último, la **matriz de asignación de recursos** es un diagrama de dispersión que enfrenta la gravedad del riesgo (eje horizontal) con el volumen de alumnado (eje vertical) para cada centro, permitiendo identificar visualmente qué centros combinan alto riesgo con gran número de alumnos y, por tanto, representan la mayor prioridad de intervención.

3.3.5 Ranking de centros

En la parte inferior de la vista se muestra el **ranking completo de centros**, ordenado de mayor a menor índice de riesgo. Cada tarjeta de centro presenta el nombre, el tipo, el número de alumnos, el índice de riesgo con indicador visual de color, y la variación respecto al curso anterior expresada en puntos. Al pulsar sobre una tarjeta, se activa el drilldown hacia la vista Micro con ese centro preseleccionado en los filtros globales.

3.4 Vista Micro: Análisis de Centro

La vista *Análisis de Centro* es la herramienta de diagnóstico causal del sistema. Se accede a ella bien directamente desde el menú, bien mediante el drilldown desde la vista Macro al seleccionar uno o varios centros concretos. Su propósito es responder a la pregunta de *por qué* un centro presenta el índice de riesgo que tiene, desagregando los factores que lo componen y permitiendo al técnico fundamentar con evidencia la naturaleza de los recursos que deben asignarse.

3.4.1 Indicadores de alumnado con comparativa interanual

La sección superior presenta **diez tarjetas KPI**, una por cada tipo de alerta, que miden el número de alumnos de los centros seleccionados que presentan ese factor de riesgo en el curso actual (T0), junto con el dato del curso anterior (T-1). Esta comparativa interanual permite evaluar si la situación ha mejorado o empeorado respecto al año precedente, aportando una dimensión temporal al diagnóstico. Las métricas son las mismas diez que en la vista Macro, pero medidas ahora a escala de alumno individual en lugar de a escala de centro.

Cada tarjeta es interactiva: al pulsarla, la tabla de alumnos de la parte inferior se filtra automáticamente para mostrar únicamente los estudiantes que presentan esa alerta, y la vista se desplaza suavemente hasta la tabla. Este mecanismo de *cross-filtering* permite pasar en un solo clic de la lectura del KPI a la lista nominal de alumnos afectados.

3.4.2 Gráficos de análisis de alumnado

Ocho gráficos ofrecen distintas perspectivas sobre la composición del riesgo en los centros seleccionados. La **evolución histórica** muestra la tendencia del riesgo medio de abandono

en los últimos cinco cursos académicos. La **distribución de suspensos** es un histograma que muestra cuántos alumnos acumulan 0, 1, 2, 3 o más asignaturas suspensas. La **distribución de riesgo** es un gráfico circular que clasifica al alumnado en los tres niveles de riesgo (Bajo, Medio, Alto). El **riesgo por ciclo** compara el porcentaje de alumnado en riesgo crítico desglosado por ciclo educativo, revelando en qué etapas se concentra el problema.

Los cuatro gráficos restantes analizan los factores socioeconómicos y familiares: la **distribución del nivel de renta** muestra la proporción de familias en cada tramo (Muy Baja, Baja, Media/Alta); el gráfico de **renta frente a suspensos** es un diagrama de dispersión que evidencia la correlación entre vulnerabilidad económica y rendimiento académico; el gráfico de **brecha digital frente a aprobados** relaciona el acceso a dispositivos e internet con la tasa de aprobado; y el gráfico de **nivel educativo familiar** muestra la distribución del máximo nivel de estudios alcanzado por los progenitores. Estos cuatro indicadores son especialmente relevantes para determinar si los recursos que necesita un centro son de naturaleza académica (refuerzo curricular) o socioeconómica (becas, dotación tecnológica, programas de acompañamiento familiar).

3.4.3 Directorio de alumnos

En la parte inferior de la vista se encuentra el **Directorio de Alumnos**, una tabla paginada (25 registros por página) que lista a todos los estudiantes que cumplen los filtros activos. Para cada alumno se muestra el número de expediente, el centro educativo y ciclo al que pertenece, la tasa de aprobado, la puntuación de riesgo de abandono con indicador visual de nivel (Bajo, Medio o Alto), y los *badges* de alertas activas (Brecha Digital, Renta Baja, Repetidor, Discapacidad o Compensatoria).

La tabla permite ordenación interactiva por cualquiera de sus columnas, tanto ascendente como descendente, e incorpora un filtro rápido multicriterio por tipo de alerta que puede combinarse con los filtros globales del sidebar. Un contador en tiempo real informa del total de alumnos que coinciden con la selección activa.

3.5 Gestión de usuarios

La sección de **Gestión de Usuarios** es accesible únicamente para cuentas con rol de administrador. Permite mantener el directorio de cuentas de acceso a la aplicación mediante las operaciones habituales de creación, edición y eliminación de usuarios. El sistema aplica reglas de protección sobre la cuenta del superadministrador (identificador 1): no puede ser eliminada, degradada a rol de visualizador ni modificada por otros administradores, garantizando que siempre exista al menos una cuenta con acceso completo al sistema.

3.6 Casos de uso principales

A continuación se describen los flujos de trabajo más representativos que ilustran cómo la herramienta apoya el proceso de toma de decisiones en la distribución de recursos educativos.

3.6.1 Caso de uso 1: Priorización de centros para asignación de recursos de apoyo

Un técnico de la administración educativa autonómica accede a la vista *Centro de Mando* al inicio del curso académico. Selecciona el curso en el filtro del sidebar y observa que la media del índice de riesgo autonómico ha subido 0,4 puntos respecto al curso anterior y que hay tres centros con índice crítico (≥ 7). Pulsa sobre la tarjeta de alerta *Repetidores 1.º/2.º de Primaria* para filtrar el ranking por esa métrica. Identifica dos centros CEIP que concentran la mayor proporción de alumnos repetidores en ciclos iniciales. Hace clic sobre la tarjeta de uno de ellos para realizar el drilldown hacia la vista Micro.

En la vista Micro, con ese centro preseleccionado, el técnico observa que el KPI de *Bajo Nivel de Estudios Familiar* es el más elevado y que el gráfico de *Nivel Educativo Familiar* confirma que más del 60% de las familias no ha completado la educación secundaria. Con esta evidencia, concluye que el recurso adecuado no es refuerzo curricular sino un programa de acompañamiento familiar, y fundamenta así su propuesta de asignación ante el órgano competente.

3.6.2 Caso de uso 2: Diagnóstico de la brecha digital en centros de zona rural

El técnico filtra la vista Macro por tipo de centro CEIP y observa en el mapa que varios centros de zonas rurales presentan marcadores rojos. Selecciona el conjunto de esos centros mediante el filtro de centro educativo y navega a la vista Micro. El KPI de *Brecha Digital* muestra un número significativamente superior al de la media autonómica, y el gráfico de *Brecha Digital frente a Aprobados* evidencia una correlación negativa clara entre la falta de dispositivos en el hogar y la tasa de aprobado. Esta información sirve de base para solicitar la dotación de equipos informáticos y conectividad en el marco de un programa de reducción de la brecha digital.

3.6.3 Caso de uso 3: Evaluación del impacto de una intervención previa

Un inspector de educación desea evaluar el impacto de un programa de refuerzo implantado hace dos cursos en un conjunto de centros de una comarca. Selecciona esos centros en el filtro de centro educativo y consulta el gráfico de *Evolución del Riesgo Autonómico* en la vista Macro. La línea temporal muestra una reducción sostenida del índice de riesgo medio

en los dos últimos cursos para esa selección. A continuación, realiza el drilldown a la vista Micro y comprueba que el KPI de *Alumnos con Suspensos* ha descendido en T0 respecto a T-1, mientras que el de *Alto Riesgo de Abandono* se mantiene estable. Esta lectura combinada permite concluir que el programa ha tenido efecto sobre el rendimiento académico inmediato pero que persisten factores estructurales de riesgo que requieren intervenciones adicionales de carácter socioeconómico.

3.6.4 Caso de uso 4: Análisis comparativo por tipo de centro

Un técnico de planificación educativa necesita determinar si los centros de Formación Profesional presentan un perfil de riesgo diferenciado respecto a los institutos de educación secundaria. Filtra la vista Macro por tipo de centro y alterna entre IES y ciclos de FP, comparando el gráfico de *Comparativa por Tipo de Centro* y la distribución de alertas en el panel institucional. Observa que los centros de FP concentran una mayor proporción de alumnado con desfase académico por edad y con riesgo socioeconómico alto, lo que orienta la planificación de recursos específicos para ese tipo de centro en el presupuesto del siguiente ejercicio.

Metodología y planificación

EL desarrollo de un proyecto de software de la envergadura de un Trabajo de Fin de Grado requiere una organización rigurosa para garantizar el cumplimiento de los plazos y la calidad de los entregables.

4.1 Metodología de desarrollo

Debido a la naturaleza investigadora y de desarrollo individual de este proyecto, no se ha adoptado una metodología ágil estricta (como Scrum), que está más orientada a equipos de desarrollo con roles definidos.

En su lugar, se ha optado por un enfoque *iterativo e incremental*. Este enfoque permite dividir el ciclo de vida del proyecto en ciclos más cortos y manejables (iteraciones), en cada uno de los cuales se añade funcionalidad o se refinan las características existentes.

Las fases iterativas de este proyecto han sido las siguientes:

1. **Análisis e investigación teórica:** Definición de los factores de riesgo de abandono escolar y estudio de la viabilidad de extracción de datos.
2. **Diseño de la arquitectura:** Elección de la pila tecnológica y diseño del ecosistema de contenedores Docker.
3. **Modelado de datos:** Diseño del esquema en estrella y mapeo origen-destino.
4. **Desarrollo ETL:** Implementación iterativa de los pipelines (extracción de tablas transaccionales, transformación y carga dimensional).
5. **Desarrollo Backend y Frontend:** Exposición de los datos mediante una API RESTful y creación de los *dashboards* visuales, ajustando los umbrales de alerta en base a los resultados observados.

Esta estrategia ha dotado al desarrollo de la flexibilidad necesaria para ajustar las reglas de negocio, como las puntuaciones de riesgo por necesidades educativas especiales o los umbrales socioeconómicos, a medida que se profundizaba en la casuística de los datos reales.

4.2 Gestión y control de versiones

Para el correcto seguimiento de la evolución del proyecto, se ha empleado **Git** como sistema de control de versiones descentralizado. El repositorio central se ha alojado en GitHub, lo que ha facilitado mantener un histórico organizado de los cambios estructurales.

Siguiendo las mejores prácticas de la industria, el repositorio se estructuró monorepo separando claramente los dominios funcionales en directorios independientes (*api*, *dwh*, *hop*, *frontend*), lo que permitió aplicar la metodología iterativa sobre componentes aislados sin interferir con el resto del ecosistema.

4.3 Planificación temporal

Para el control y la gestión del tiempo del proyecto, se ha diseñado una planificación estructurada en un horizonte temporal de cuatro meses, abarcando desde el inicio de la investigación hasta la defensa final. Esta planificación se ha plasmado visualmente mediante un diagrama de Gantt (ver Figura 4.1), que permite monitorizar el solapamiento y la dependencia entre las distintas iteraciones de desarrollo.

El cronograma se divide en cinco grandes bloques de trabajo:

- **Fase 1: Análisis y diseño (Mes 1).** Dedicada a la investigación teórica del problema, la definición de requisitos funcionales y el diseño de la arquitectura tecnológica y el modelo de datos.
- **Fase 2: Ingeniería de datos (Mes 2).** Centrada en el despliegue del entorno Docker y el desarrollo intensivo de los procesos ETL con Apache Hop.
- **Fase 3: Desarrollo Backend (Mes 3).** Construcción de la API RESTful con FastAPI y las consultas analíticas sobre el Datamart.
- **Fase 4: Desarrollo Frontend y validación (Mes 3–4).** Implementación del cuadro de mando interactivo en React y ejecución de pruebas globales de integración del sistema completo.
- **Fase 5: Documentación y cierre (Mes 4).** Redacción de la presente memoria académica, revisión de entregables y preparación de la defensa oficial.

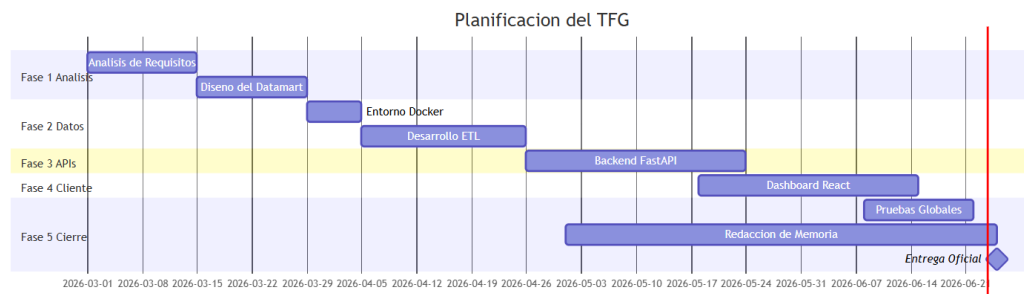


Figura 4.1: Diagrama de Gantt de la planificación del proyecto.

4.4 Presupuesto y costes

Todo proyecto de ingeniería de software conlleva un consumo de recursos que debe ser cuantificado para evaluar su viabilidad económica. En este apartado se presenta una estimación del presupuesto del proyecto, dividida en costes de recursos humanos y costes materiales o de infraestructura.

4.4.1 Recursos humanos

El principal coste de este Trabajo de Fin de Grado radica en el tiempo invertido en su investigación, desarrollo y documentación. Para realizar una estimación realista, se ha calculado el coste equivalente en un entorno profesional, asumiendo el rol de un Ingeniero de Software Júnior para el desarrollo, y el rol de un Arquitecto/Director de Proyecto para las labores de tutorización.

Tomando como referencia los salarios medios del sector tecnológico en España, se ha establecido un coste por hora de 20 € para el perfil júnior y de 50 € para el perfil sénior (tutor). La estimación de horas se ajusta a la carga de trabajo típica de un TFG de esta envergadura (equivalente a 12 créditos ECTS, aproximadamente 300–400 horas de trabajo autónomo).

Tabla 4.1: Estimación de presupuesto de recursos humanos.

Rol / Puesto	Coste/hora	Horas trabajadas	Coste total
Programador / Ingeniero Júnior	20 €	400	8.000 €
Tutor / Director de Proyecto	50 €	15	750 €
Total estimado			8.750 €

4.4.2 Recursos materiales e infraestructura

Una de las decisiones arquitectónicas fundamentales de este proyecto ha sido la adopción exclusiva de tecnologías *Open Source* (PostgreSQL, Apache Hop, FastAPI, React) y el uso de contenedores Docker. Esta estrategia garantiza que la solución pueda ser desplegada y auditada sin incurrir en costes de licenciamiento de software privativo.

Durante la fase de desarrollo, el sistema se ha ejecutado íntegramente en un entorno local, utilizando el equipo informático del autor (cuyo coste de amortización se asume implícito). No obstante, para un despliegue en producción de la herramienta accesible vía web, sería necesario contar con una infraestructura de alojamiento y un dominio seguro. A modo de referencia, se presenta una estimación de los costes de infraestructura mínimos para el primer año de operación en un entorno *Cloud* básico:

Tabla 4.2: Estimación de costes materiales y de infraestructura (1 año).

Concepto	Descripción	Coste estimado
Licencias de Software	PostgreSQL, Apache Hop, React, FastAPI	0,00 €
Dominio web (.es)	Registro anual para acceso público y certificados SSL	12,00 €
Servidor Virtual (VPS)	Instancia básica en la nube (ej. 4 GB RAM, 2 vCPU) a ~15 €/mes	180,00 €
Total estimado (1 año)		192,00 €

En conclusión, el proyecto demuestra una alta viabilidad económica. Al carecer de costes de licencias, la administración pública u organización educativa que desee adoptar esta solución únicamente tendría que asumir los costes de alojamiento en sus propios servidores (o en una nube pública) y el mantenimiento evolutivo del código.

Análisis y arquitectura

EN este capítulo se detallan los requisitos del sistema y la arquitectura tecnológica seleccionada para la implementación del Datamart y el cuadro de mando. Es importante destacar que todo el proyecto se ha diseñado y desarrollado utilizando exclusivamente tecnologías **Open Source** (código abierto). Esta decisión estratégica garantiza que cualquier centro educativo, sea público o privado, pueda adoptar y desplegar esta solución sin incurrir en costes de licenciamiento de software comercial.

5.1 Requisitos del sistema

Para lograr el objetivo de detectar de forma temprana el riesgo de abandono escolar y la brecha digital, el sistema debe cumplir con los siguientes requisitos funcionales y no funcionales:

- **RF1 - Extracción y Transformación:** El sistema debe ser capaz de extraer datos en bruto del sistema transaccional (PostgreSQL), limpiarlos y aplicar reglas de negocio para calcular índices de riesgo socioeconómico y académico.
- **RF2 - Modelo Dimensional:** Los datos deben estructurarse en un Data Warehouse bajo un modelo de estrella para facilitar consultas analíticas ultrarrápidas y cruces demográficos.
- **RF3 - Visualización y Alertas:** Se debe proveer una interfaz web (Dashboard) que muestre de forma intuitiva qué centros educativos o cohortes geográficas presentan un riesgo alto de vulnerabilidad sistémica y fracaso escolar.
- **RNF1 - Modularidad y Escalabilidad:** La arquitectura debe estar dividida en microservicios independientes, facilitando su despliegue y mantenimiento.

- **RNF2 - Tecnologías Libres:** Obligatoriedad de utilizar herramientas Open Source en todas las capas del proyecto.

5.2 Análisis del sistema transaccional origen

Para comprender la necesidad de construir un Data Warehouse y diseñar los flujos ETL, es fundamental analizar previamente la estructura de la base de datos origen. En este proyecto se ha simulado una base de datos transaccional (OLTP) en PostgreSQL que replica el sistema de gestión académica y administrativa estándar de los centros educativos.

Este sistema está altamente normalizado para garantizar la integridad referencial y evitar la redundancia de datos en las operaciones diarias (altas de matrículas, registro de calificaciones, encuestas familiares). El modelo relacional consta de 14 tablas principales interconectadas (ver Figura 5.1 y Figura 5.2), agrupadas en los siguientes dominios lógicos:

- **Dominio Académico:** Tablas *CICLO_EDUCATIVO*, *ESPECIALIDAD*, *CURSO* y *ASIGNATURA*. Definen el catálogo formativo del sistema.
- **Dominio de Matriculación y Evaluación:** Tablas *CENTRO*, *ADMISION*, *MATRICULA*, *ASIGNATURA_MATRICULA* y la entidad débil *LINEA_EXPEDIENTE*, que almacena el histórico de calificaciones por evaluación.
- **Dominio del Alumnado y Diversidad:** Tablas *ESTUDIANTE*, *ADAPTACION_CURRICULAR* y *ADAPTACION_ASIGNATURA*, registrando los expedientes y las medidas de apoyo educativo.
- **Dominio Socioeconómico:** Tablas *RESPONSABLE_LEGAL* y *ENCUESTA*. Contienen el histórico de ingresos, acceso a internet y nivel educativo de los progenitores; datos vitales para calcular posteriormente el riesgo socioeconómico.

Dada la alta fragmentación de la información (por ejemplo, para relacionar las calificaciones de un estudiante con el nivel de renta de sus tutores es necesario cruzar simultáneamente las tablas *LINEA_EXPEDIENTE*, *ASIGNATURA_MATRICULA*, *MATRICULA*, *ESTUDIANTE*, *RESPONSABLE_LEGAL* y *ENCUESTA*), ejecutar algoritmos analíticos directamente sobre este esquema penalizaría severamente el rendimiento del sistema en producción. Esta limitación inherente a los sistemas OLTP justifica plenamente el diseño dimensional del Datamart.

5.3 Arquitectura tecnológica basada en Docker

Para cumplir con el requisito de modularidad, la arquitectura del proyecto se ha construido bajo el paradigma de contenedores utilizando **Docker** [8]. Esto permite aislar cada com-

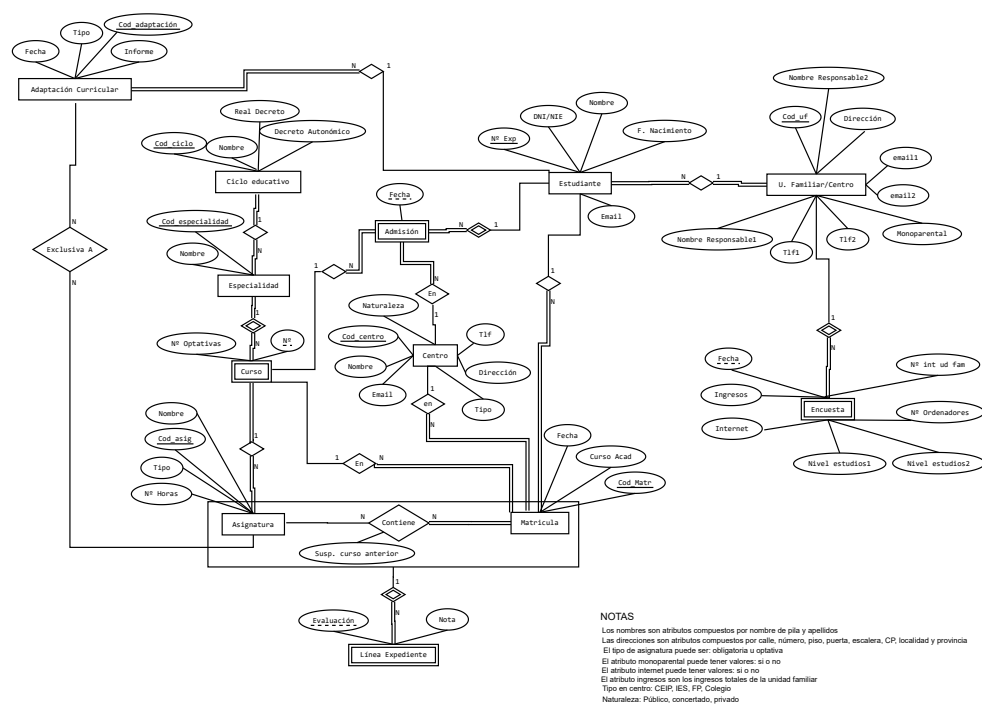


Figura 5.1: Modelo Entidad-Relación (ER) de la base de datos transaccional origen.

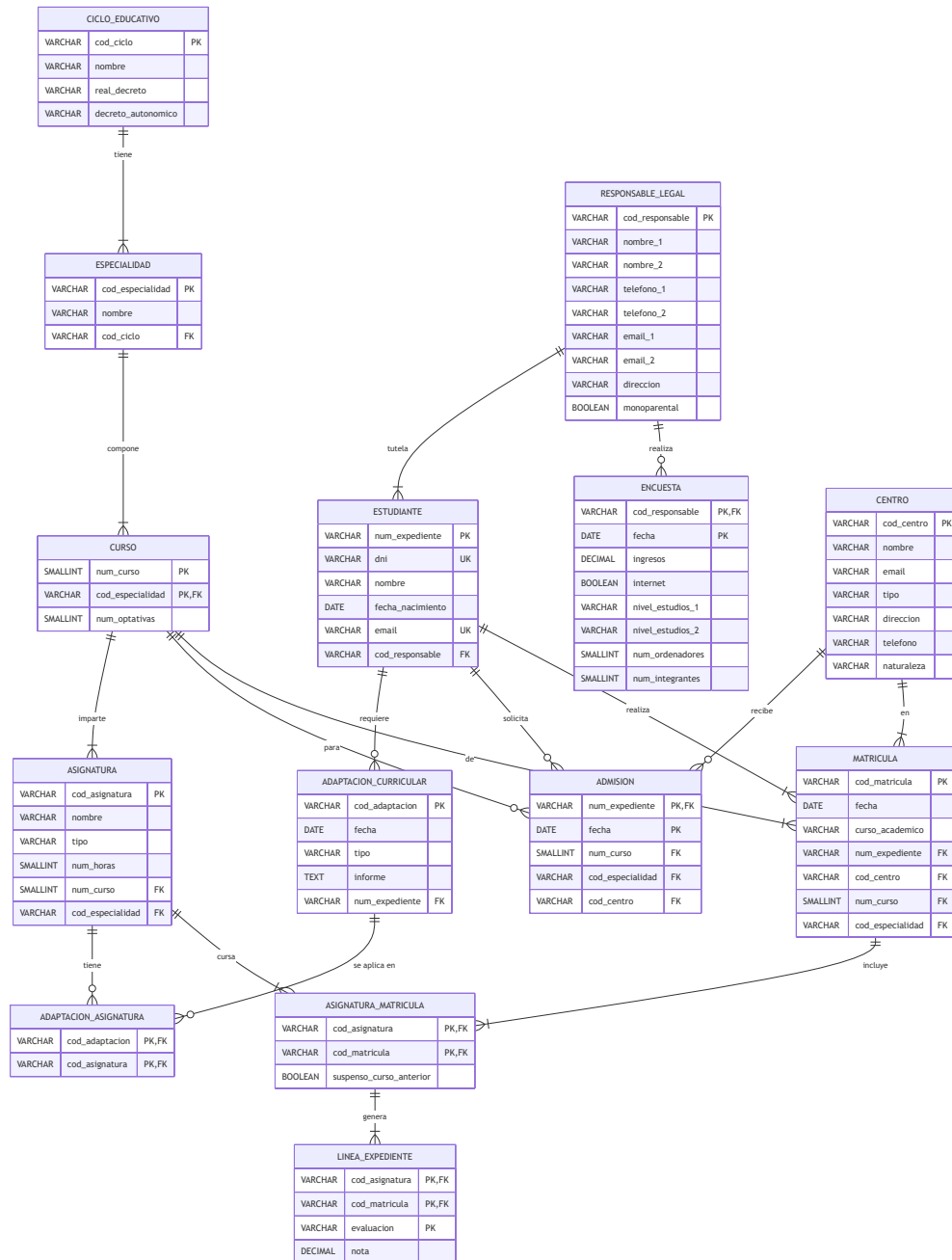


Figura 5.2: Esquema Relacional de origen.

ponente del sistema en su propio entorno, asegurando que funcionen de manera predecible independientemente del servidor donde se desplieguen.

El ecosistema está compuesto por seis contenedores principales orquestados mediante *docker-compose*:

1. **Origen de Datos Transaccional (PostgreSQL) [9]**: Simula la base de datos operativa (*source*) de los centros educativos. Contiene tablas normalizadas con los expedientes, matrículas, notas y encuestas familiares en tiempo real.
2. **Data Warehouse Analítico (PostgreSQL)**: Actúa como el repositorio central de BI (*dwh*). Almacena los datos estructurados en un modelo dimensional (esquema en estrella) optimizado para lectura y agregación, conteniendo el historial acumulado.
3. **Orquestador ETL (Apache Hop Web)**: Contenedor que ejecuta los flujos de transformación (*pipelines*). Se encarga de conectarse al Origen de Datos, extraer la información, calcular las métricas de riesgo (socioeconómico y de abandono) y cargar el resultado en el Data Warehouse.
4. **Servidor Backend RESTful (FastAPI) [10]**: Construido en Python, este contenedor expone una API segura que se conecta al Data Warehouse, realiza las consultas analíticas pesadas mediante *SQLAlchemy* y sirve los resultados en formato JSON al cliente.
5. **Aplicación Cliente Dashboard (React/Vite) [11]**: Interfaz gráfica web moderna donde los gestores de la administración visualizan los datos. Utiliza *Recharts* para los gráficos y sigue una estética visual basada en *Glassmorphism*.
6. **Servidor Web y Proxy Inverso (Nginx)**: Se encarga de servir la aplicación React compilada y de enrutar las peticiones de red hacia la API del Backend, asegurando un único punto de entrada seguro para la aplicación.

Esta separación clara entre la base de datos transaccional (OLTP) y el almacén analítico (OLAP) garantiza que las complejas consultas del Dashboard no penalicen el rendimiento de las operaciones diarias del colegio.

5.3.1 Justificación del paradigma de contenedores

Aunque el diseño de una arquitectura distribuida conlleva una mayor carga de razonamiento inicial respecto a una solución monolítica, esta decisión aporta un enorme valor añadido al proyecto por tres motivos fundamentales:

- **Aislamiento y Reproducibilidad**: Cada contenedor empaqueta sus propias dependencias (por ejemplo, librerías concretas de Python para FastAPI o módulos de Node

para React). Esto elimina el clásico problema de *"funciona en mi máquina"* y garantiza que el comportamiento del software sea idéntico en desarrollo, pruebas y producción.

- **Despliegue Universal (*One-Click Deployment*):** Al estar orientado a centros educativos (instituciones que frecuentemente carecen de departamentos de IT especializados o presupuestos para infraestructuras complejas), la orquestación con *docker-compose* permite levantar toda la plataforma (bases de datos, ETL, API y web) con un único comando. Esto facilita enormemente su transferencia tecnológica y adopción real.
- **Topología de Red y Seguridad:** Docker permite crear redes virtuales cerradas (*Docker Networks*). De esta forma, bases de datos críticas como el Data Warehouse y los flujos de transformación de Apache Hop quedan totalmente invisibles desde el exterior. Solamente el proxy inverso (Nginx) expone los puertos estrictamente necesarios, añadiendo una capa de seguridad pasiva esencial cuando se tratan datos académicos y demográficos de menores.

Diseño del datamart analítico

EL diseño del Datamart constituye el núcleo lógico de este proyecto. Se ha optado por un modelo dimensional en estrella (Star Schema) basado en la metodología de Ralph Kimball, pero introduciendo optimizaciones específicas para tratar variables demográficas de alta cardinalidad e historificación de expedientes.

6.1 Modelo dimensional en estrella

A diferencia de las bases de datos transaccionales, que están altamente normalizadas, el esquema del Data Warehouse se ha desnormalizado en dos tipos de tablas: **Tablas de Hechos** (que almacenan métricas y claves foráneas) y **Tablas de Dimensiones** (que contienen descriptores textuales para filtrar y agrupar).

En nuestro diseño final se han establecido dos tablas de hechos de diferente granularidad para responder a distintas necesidades analíticas, rodeadas por un total de siete dimensiones. La Figura 6.1 muestra el esquema completo del Datamart, con todas las tablas del esquema *dwh*, sus columnas, claves primarias, claves foráneas y las relaciones entre ellas.

6.1.1 Tablas de dimensiones en detalle

Las dimensiones contextualizan los datos numéricos. Para este proyecto se han diseñado las siguientes:

- ***dim_estudiante* (SCD Tipo 2)**: Contiene los datos personales del alumno y su responsable legal. Para reflejar la inestabilidad habitacional o cambios de tutela (muy comunes en entornos vulnerables), se implementó una *Dimensión de Variación Lenta de Tipo 2* (SCD2). Esto significa que ante un cambio de tutor o domicilio, no se sobrescribe la fila, sino que se caduca (marcando una fecha de fin) y se crea una nueva, preservando el contexto histórico de las notas de años anteriores.

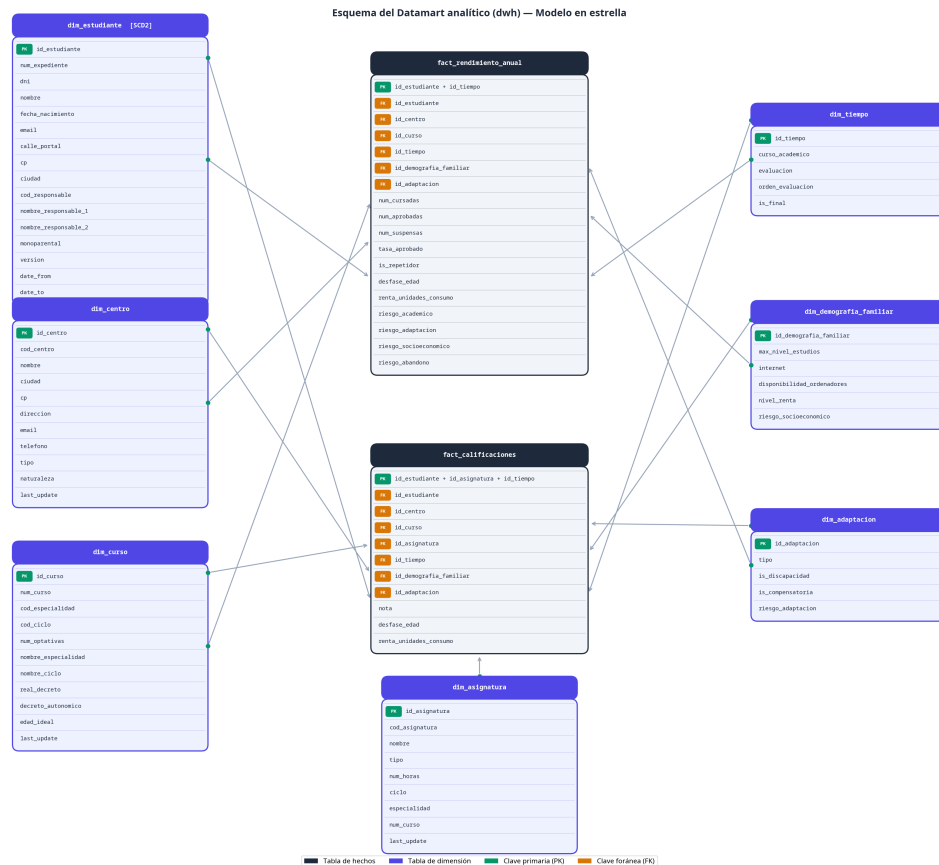


Figura 6.1: Esquema del Datamart analítico (*dwh*) en modelo estrella: dos tablas de hechos de distinta granularidad (*fact_rendimiento_anual* y *fact_calificaciones*) rodeadas por siete dimensiones. Se indica con [SCD2] la dimensión *dim_estudiante*, que implementa una Dimensión de Variación Lenta de Tipo 2.

- ***dim_centro***: Almacena la información institucional y geográfica de los colegios e institutos.
- ***dim_curso***: Consolida la estructura académica (Ciclo, Especialidad y Curso).
- ***dim_asignatura***: Catálogo de materias independientes del currículo, con su carga horaria.
- ***dim_tiempo***: Dimensión temporal que almacena el año académico y el periodo de evaluación.
- ***dim_demografia_familiar***: Utiliza un modelo demográfico híbrido para reducir la cardinalidad. En lugar de almacenar ingresos exactos, clasifica a las familias por su nivel de renta en tramos, conectividad a internet y disponibilidad de ordenadores.
- ***dim_adaptacion***: Registra si el alumno cuenta con adaptaciones curriculares, categorizándolas por discapacidad médica o compensación sociopedagógica.

6.1.2 Tablas de hechos

El modelo implementa dos niveles de análisis a través de dos tablas de hechos:

- ***fact_calificaciones* (Granularidad Atómica)**: Representa el nivel más detallado. Cada fila es la nota exacta de un estudiante en una asignatura durante una evaluación concreta. Es útil para rastrear el progreso pormenorizado a lo largo del curso.
- ***fact_rendimiento_anual* (Granularidad Agregada)**: Cada fila representa el resumen académico total de un alumno en un curso completo. Consolida el número de asignaturas matriculadas, aprobadas, la tasa de éxito, las asignaturas arrastradas de años anteriores y si el alumno ha repetido. Es en esta tabla donde se calculan de manera definitiva los KPIs críticos y el índice de Riesgo de Abandono Escolar Temprano.

6.2 Reglas de negocio y cálculo de riesgos

Para que el Dashboard ofrezca métricas accionables a nivel autonómico, se han parametrizado dos indicadores de riesgo fundamentales mediante sistemas de puntuación acumulativa, integrados en la lógica del Datamart.

6.2.1 Riesgo socioeconómico

Este riesgo evalúa el contexto familiar de exclusión y brecha digital. Los puntos se calculan sumando los siguientes factores:

- **Estudios de los Progenitores (Estándar PARED):** Se ha adoptado el enfoque de la OCDE en sus informes PISA, donde el riesgo correlaciona con el mayor nivel educativo alcanzado por cualquiera de los padres [12]. Puntuaciones aplicadas a cada progenitor: Sin estudios de primaria (5 pts), Primaria (3 pts), Secundaria o superior (1 pt).
- **Renta por Unidades de Consumo (Escala OCDE Modificada):** Para determinar el riesgo económico no se divide la renta de forma puramente lineal (per cápita). Siguiendo la metodología AROPE de Eurostat y las directrices del INE [13, 14], la escala teórica estipula que el primer adulto computa como 1 unidad de consumo, los integrantes mayores de 14 años como 0,5 unidades y los menores de 14 años como 0,3. No obstante, debido a que el sistema transaccional de origen no recopila la edad exacta del resto de convivientes (hermanos o familiares), ha sido necesario aplicar una adaptación metodológica en la ETL. Se asigna un peso de 1 al primer integrante y de 0,5 al resto, independientemente de su edad. En base a los ingresos anuales equivalentes obtenidos tras la ponderación, se establecen los siguientes niveles de renta en la capa de transformación:
 - **Muy Baja (Pobreza severa):** Ingresos por unidad de consumo inferiores a 7.000 €/año. Supone una alta penalización (15 pts).
 - **Baja (Umbral de pobreza):** Ingresos por unidad de consumo entre 7.000 € y 11.000 €/año. Supone una penalización moderada (7 pts).
 - **Media/Alta:** Ingresos por unidad de consumo superiores a 11.000 €/año. Ambas categorías se han unificado ya que no suponen un factor de riesgo económico y, por tanto, no suman puntos (0 pts).
- **Brecha Digital (Conectividad):** Sin acceso a Internet en el hogar (3 pts).
- **Brecha Digital (Hardware):** 0 ordenadores (5 pts), menos de 1 ordenador por hijo (2 pts).

Nota: Si la suma del riesgo socioeconómico supera los 16 puntos, la familia se considera en situación de alta vulnerabilidad.

6.2.2 Riesgo de abandono del alumno

Combina factores académicos, de apoyo y de arrastre socioeconómico para emitir una alerta crítica global:

- **Carga del Riesgo Socioeconómico:** De 16 a 21 puntos acumulados familiares (5 pts). Más de 21 puntos (10 pts).

- **Penalización Académica Temprana (1º o 2º de Primaria):** Los fracasos en ciclos iniciales marcan un déficit crónico. Suspenso en evaluación regular (10 pts), asignatura suspensa a final de curso (20 pts), repetición del curso (40 pts).
- **Penalización Académica General (Resto de cursos):** Suspenso en evaluación regular (1 pt), asignatura suspensa a final de curso (5 pts), repetición (10 pts).
- **Necesidades de Apoyo y Adaptación:** Adaptación por motivos de discapacidad (10 pts). Adaptación por otros motivos o compensatoria (5 pts).

Estos valores numéricos quedan consolidados en *fact_rendimiento_anual*, lo que permite al Backend (FastAPI) entregar al Dashboard de forma instantánea el listado de centros con mayor concentración de alumnado en riesgo inminente de abandono.

Capa ETL con Apache Hop

EL éxito de un sistema de Business Intelligence radica en la calidad, consistencia y fiabilidad de sus datos. En esta fase central del proyecto se ha utilizado **Apache Hop**, una plataforma moderna e innovadora de orquestación y procesamiento de datos basada en metadatos, para diseñar e implementar los procesos de Extracción, Transformación y Carga (ETL).

Este capítulo detalla la arquitectura de los flujos de datos (*Pipelines*) y de trabajo (*Workflows*), exponiendo las decisiones técnicas tomadas para poblar el Datamart y calcular algorítmicamente los índices de riesgo de abandono escolar.

7.1 Configuración del entorno y conexiones

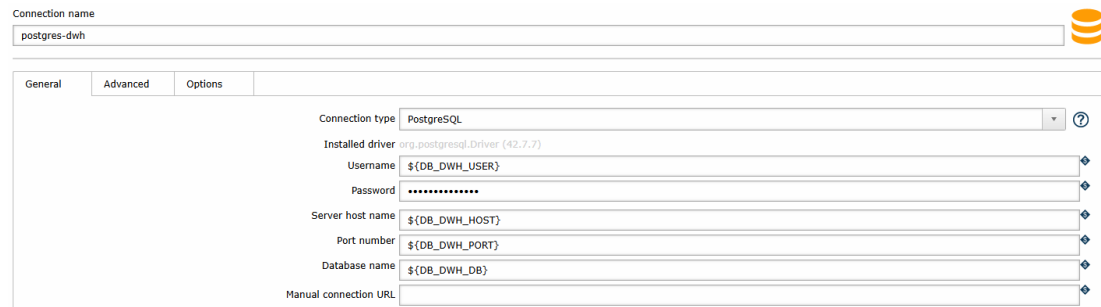
El ecosistema de Apache Hop se ha desplegado utilizando su imagen oficial de Docker (*apache/hop-web*), garantizando un entorno de ejecución inmutable. La parametrización de las conexiones a bases de datos se ha resuelto de forma dinámica utilizando variables de entorno de la JVM pasadas en el momento de arranque del contenedor (e.g., *-DDB_SOURCE_HOST*).

En la interfaz de Hop, se han configurado dos conexiones relacionales mediante el driver JDBC de PostgreSQL (ver Figura 7.1):

- **conn_source**: Conexión de solo lectura apuntando a la base de datos transaccional simulada.
- **conn_dwh**: Conexión de lectura y escritura apuntando al esquema dimensional del Data Warehouse.

7.2 Análisis de pipelines

A continuación, se detalla el ciclo de vida del dato desglosando el funcionamiento técnico de cada uno de los pipelines (*.hpl*) que componen el sistema ETL. Un pipeline en Apache Hop



The screenshot shows the 'General' tab of the connection configuration for 'postgres-dwh'. The 'Connection type' is set to 'PostgreSQL'. The 'Installed driver' is 'org.postgresql.Driver (42.7.7)'. The 'Username' is '\$(DB_DWH_USER)', 'Password' is masked with dots, 'Server host name' is '\$(DB_DWH_HOST)', 'Port number' is '\$(DB_DWH_PORT)', and 'Database name' is '\$(DB_DWH_DB)'. There is a 'Manual connection URL' field at the bottom.

Figura 7.1: Configuración de la conexión *postgres-dwh*.

es un grafo dirigido acíclico (DAG) donde los nodos (*Transforms*) ejecutan operaciones específicas (lectura, filtrado, cálculos) y las aristas (*Hops*) canalizan el flujo de registros directamente en memoria.

7.2.1 Volcado a staging

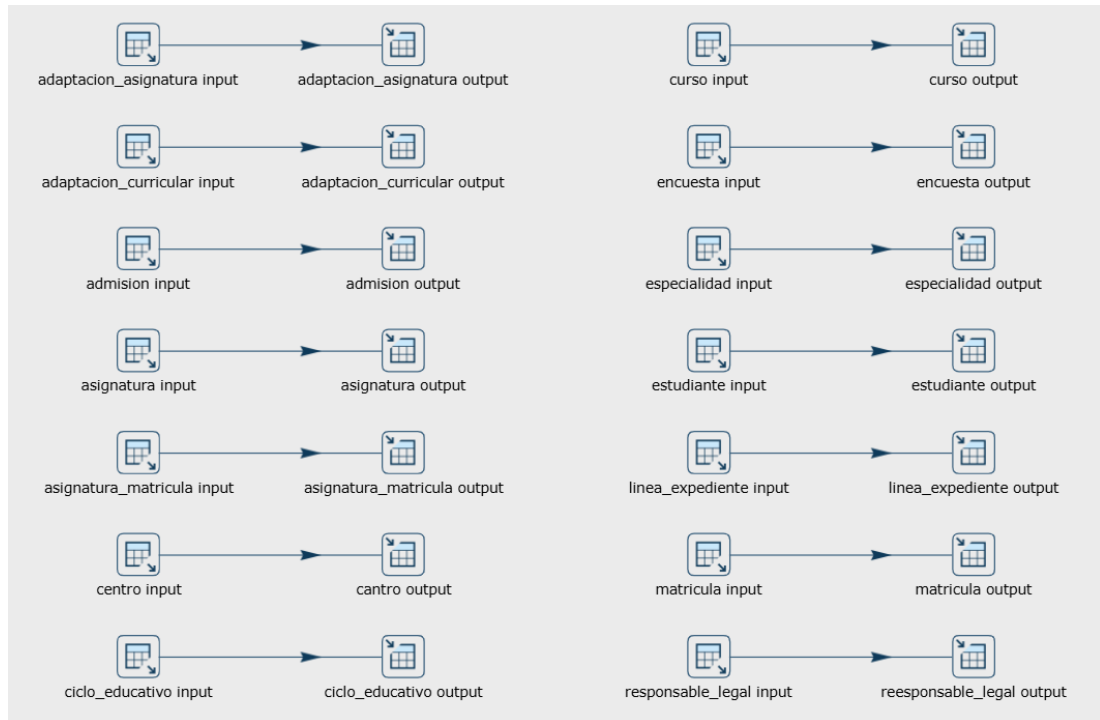
Para no sobrecargar la base de datos origen transaccional durante los cruces analíticos posteriores, el primer paso estricto de la arquitectura ETL consiste en realizar una clonación exacta de las tablas de producción hacia el esquema intermedio transitorio *raw* (Staging Area).

Como se aprecia en la Figura 7.2, el pipeline *extract_all_to_staging.hpl* establece 14 hilos independientes de tipo *Table Input* (origen) conectados a *Table Output* (destino). Este componente implementa un mecanismo de extracción incremental mediante el uso del parámetro dinámico *\$(CURSO_ACTUAL)*. Los nodos de extracción ejecutan subconsultas SQL que filtran y capturan exclusivamente los datos del curso lectivo en vigor (ej. *WHERE curso_academico = '\$(CURSO_ACTUAL)'*). Para permitir recargas históricas completas, se incluye la condición lógica *OR '\$(CURSO_ACTUAL)' = ''*, garantizando una lectura masiva si el orquestador omite el parámetro. Este diseño evita volcados completos innecesarios y minimiza drásticamente la carga de red en cada ejecución por evaluación.

Una decisión de diseño fundamental documentada en este paso es la **ausencia de claves foráneas** en el esquema destino. Al no existir restricciones referenciales en la capa *raw*, Hop puede paralelizar la inserción masiva de todas las entidades simultáneamente sin generar bloqueos lógicos por dependencias, minimizando drásticamente la ventana de tiempo de extracción.

7.2.2 Dimensión de demografía familiar

El núcleo de las reglas de negocio del proyecto reside en este pipeline. Su misión es ingerir los datos brutos de la tabla de encuestas (*raw.encuesta*) y destilarlos para construir una dimensión de baja cardinalidad que consolide los perfiles de riesgo socioeconómico. Su imple-

Figura 7.2: Flujo del pipeline *extract_all_to_staging.hpl*.

mentación técnica encadena operaciones aritméticas, lógicas condicionales y deduplicación (ver Figura 7.3):

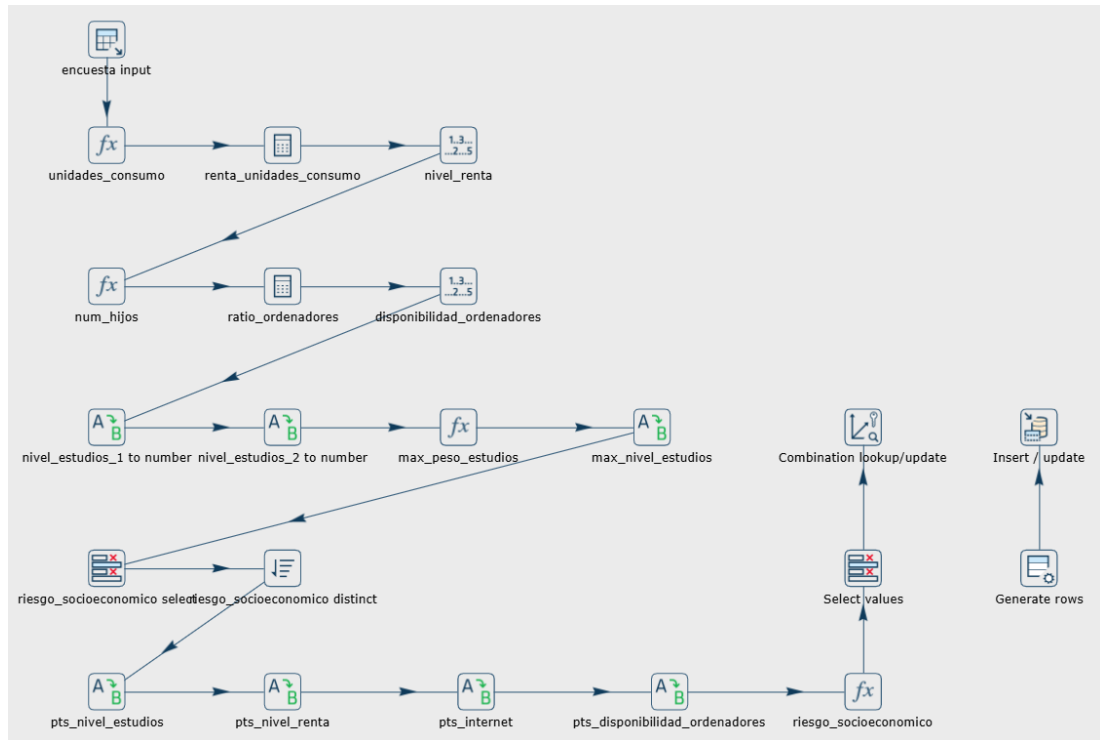
1. **Cálculo de Hijos a Cargo (*Formula*):** Dado que la tabla origen *raw.encuesta* solo informa del número total de integrantes del hogar, el pipeline calcula el número de hijos restando los adultos. Asume 1 adulto si no hay datos del segundo tutor, o 2 si los hay, garantizando matemáticamente un mínimo de 1 hijo.
2. **Cálculos de Ratio y Consumo (*Calculator*):** Se inicializan variables continuas dividiendo algebraicamente los ingresos brutos entre las unidades de consumo del hogar, obteniendo la *renta_unidades_consumo*. Paralelamente, se divide *num_ordenadores* entre *num_hijos* calculados en el paso anterior para obtener la *ratio_ordenadores*.
3. **Discretización de Variables Continuas (*NumberRange*):** Las variables calculadas se segmentan en bandas analíticas fijas:
 - *nivel_renta*: "Muy baja" (< 7.000 €), "Baja" (7.000 € - 11.000 €) o "Media/Alta" (> 11.000 €).
 - *disponibilidad_ordenadores*: "Ninguno" (0), "Menos de 1 por hijo" (0.001 a 0.999), o "1 o mas por hijo" (≥ 1).

4. **Máximo Nivel de Estudios (*ValueMapper* y *Formula*):** El pipeline transforma los niveles de estudio de ambos progenitores en un peso numérico (Universitarios=3, Secundarios=2, Primarios=1, Sin Estudios=0), aplica una función $\text{MAX}([\text{peso_estudios_1}], [\text{peso_estudios_2}])$ para quedarse con el nivel más alto del hogar, y vuelve a traducir ese número a cadena de texto (*max_nivel_estudios*).
5. **Asignación de Puntos de Riesgo (*ValueMapper*):** Se inyectan los pesos definidos por el negocio para calcular el riesgo de abandono:
 - *pts_nivel_renta*: Muy baja (15 pts), Baja (7 pts), Media/Alta (0 pts).
 - *pts_nivel_estudios*: Sin Estudios (10 pts), Primarios (6 pts), Secundarios (2 pts), Universitarios (0 pts).
 - *pts_disponibilidad_ordenadores*: Ninguno (5 pts), Menos de 1 (2 pts), 1 o más (0 pts).
 - *pts_internet*: Sin acceso (3 pts), Con acceso (0 pts).
6. **Ecuación Maestra (*Formula*):** Se materializa la variable predictiva final sumando todos los factores: $[\text{pts_disponibilidad_ordenadores}] + [\text{pts_internet}] + [\text{pts_nivel_estudios}] + [\text{pts_nivel_renta}]$.
7. **Optimización de Cardinalidad (*SortRows* - *unique_rows=Y*):** Para evitar que la dimensión crezca con cada estudiante, un nodo ordena por los atributos cualitativos y descarta los duplicados exactos en memoria. De miles de encuestas, el Data Warehouse solo almacena las combinaciones únicas posibles.
8. **Carga en Datamart (*Combination lookup/update*):** Se vuelcan los perfiles deduplicados en *dwh.dim_demografia_familiar*, generando una clave subrogada autoincremental (*id_demografia_familiar*) que será utilizada posteriormente por la tabla de hechos.

7.2.3 Dimensión de estudiante

La dimensión del alumnado requiere un modelado avanzado, ya que la casuística de un estudiante no es estática: puede cambiar de domicilio, de tutor legal o de situación familiar a lo largo de los años. Si el sistema simplemente sobrescribiese estos datos, se perdería el contexto socioeconómico real que tenía el alumno cuando obtuvo una calificación pasada. Para solventarlo, este pipeline implementa un algoritmo SCD (Slowly Changing Dimension) Tipo 2 apoyado en la limpieza previa de cadenas (ver Figura 7.4).

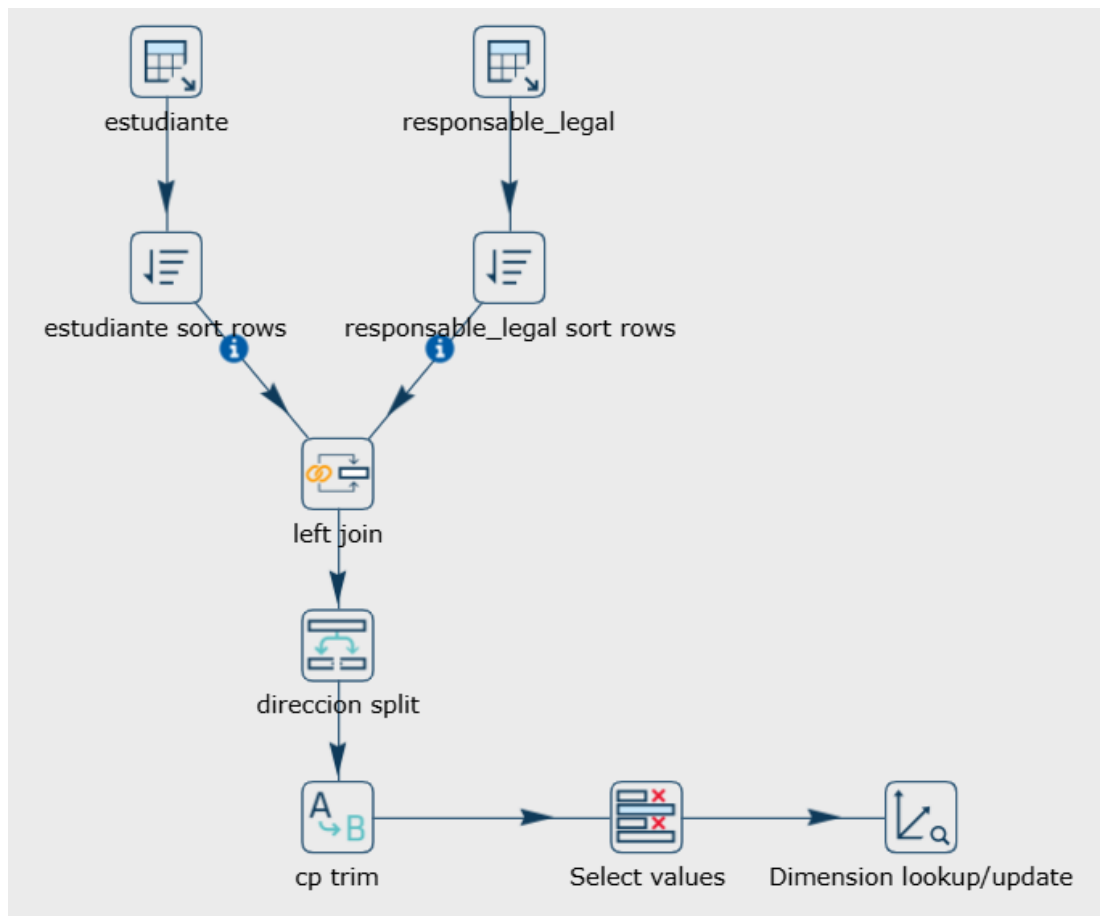
1. **Cruce y Limpieza de Cadenas (*MergeJoin* y *FieldSplitter*):** El flujo extrae asincrónicamente las entidades *raw.estudiante* y *raw.responsable_legal*, ordenándolas en memoria

Figura 7.3: Flujo del pipeline *dim_demografia_familiar.hpl*.

(*SortRows*) para ejecutar un *Merge Join* tipo LEFT OUTER por el campo *cod_responsable*. A continuación, un nodo *FieldSplitter* disecciona la cadena cruda de la dirección separándola por comas en las columnas *calle_portal*, *puerta*, *cp* y *ciudad*.

2. **Casteo de Datos (*ReplaceString* y *SelectValues*):** El nodo *ReplaceString* elimina la subcadena "CP " de los códigos postales, permitiendo que el paso subsiguiente (*SelectValues*) lo formatee analíticamente como un tipo numérico (Integer) y renombre simultáneamente las variables de los tutores a la nomenclatura final del DWH.
3. **Algoritmo SCD Tipo 2 (*Dimension lookup/update*):** El motor principal del pipeline es este nodo nativo de Hop, configurado para rastrear el ciclo de vida del *num_expediente* (Clave Natural) gestionando automáticamente una clave subrogada *id_estudiante* (autoincremental), un contador de *version* y las marcas temporales *date_from* y *date_to*. La maestría de esta configuración reside en la micro-gestión de actualizaciones campo a campo:
 - **Punch through (Comportamiento SCD Tipo 1):** Campos intrínsecos como el *dni*, *nombre* o *fecha_nacimiento*. Una corrección en origen de estos datos no justifica crear un nuevo histórico; Hop sobrescribe el valor directamente en todas las versiones pasadas de ese estudiante.

- **Insert (Comportamiento SCD Tipo 2):** Campos dependientes del contexto socioeconómico, como *cod_responsable*, *calle_portal*, *ciudad* y *monoparental*. Si Hop detecta una mutación en cualquiera de estos campos frente al registro vigente en el DWH, congela la fila antigua (estampando el *date_to* actual), e inserta una nueva fila virgen con la versión incrementada y un nuevo *id_estudiante*.
- **Update:** Datos de contacto secundarios como el *email*, que se actualizan silenciosamente en la versión activa sin generar ruido histórico.

Figura 7.4: Flujo del pipeline *dim_estudiante.hpl*.

7.2.4 Dimensión de tiempo

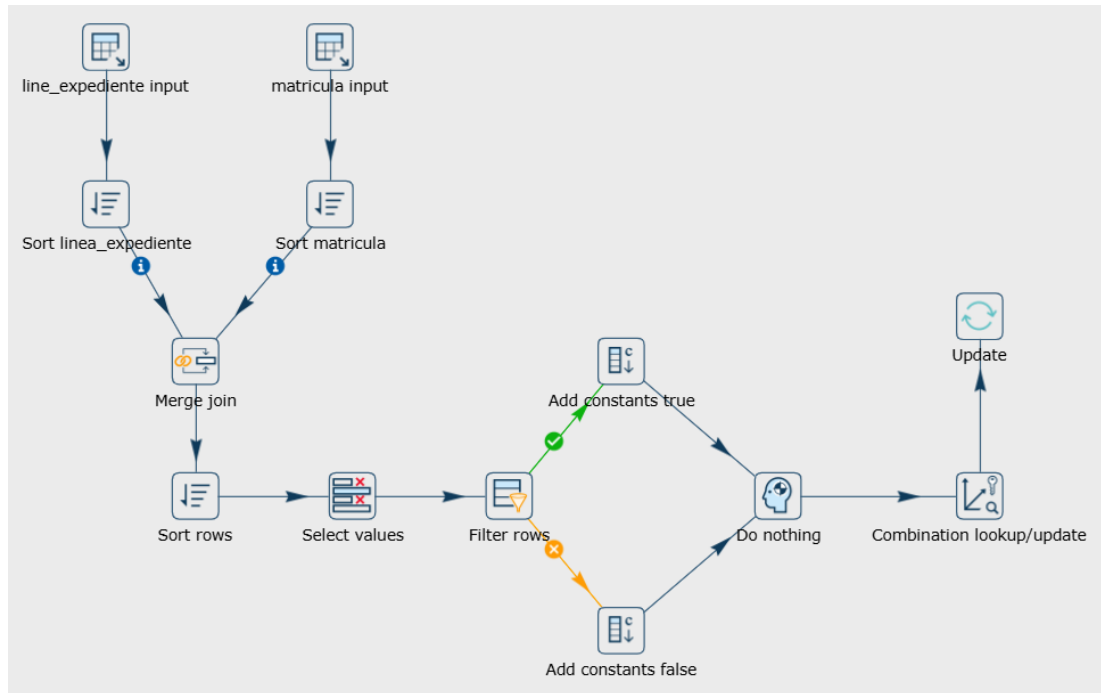
A diferencia de un calendario estándar por días o meses, este sistema requiere una dimensión de tiempo puramente académica que categorice los momentos evaluativos de un alumno (por ejemplo, "Curso 2023-2024, 1ª Evaluación"). Este pipeline infiere automáticamente estas temporalidades desde los propios registros transaccionales (ver Figura 7.5).

1. **Cruce Transaccional (*MergeJoin*):** El flujo no lee de una tabla maestra explícita, sino que extrae las operaciones transaccionales *raw.linea_expediente* y *raw.matricula*. Tras ordenar ambas por *cod_matricula* mediante nodos *SortRows*, aplica un cruce de tipo INNER JOIN para asociar la *evaluacion* en la que se puso cada nota con el *curso_academico* de la matrícula correspondiente.
2. **Extracción del Dominio (*SortRows - unique_rows=Y*):** De los cientos de miles de líneas de expediente cruzadas, se extrae el dominio temporal real filtrando únicamente los pares únicos de *curso_academico* y *evaluacion*, purificando así la dimensionalidad.
3. **Análisis Léxico Condicional (*FilterRows*):** Un paso crítico para el negocio es aislar si una evaluación es la definitiva o una parcial (fundamental para el pipeline posterior de *fact_rendimiento_anual*). Un nodo *FilterRows* analiza léxicamente el texto de la evaluación buscando la subcadena "Final" (evaluando múltiples casuísticas mediante condicionales *CONTAINS* encadenados con operadores OR: "Final", "final", "FINAL").
4. **Bifurcación de Flujo Dirigido:** Dependiendo del resultado del filtro anterior, Hop enruta el dato físicamente por dos caminos paralelos. Si el filtro resulta verdadero, el registro se desvía a un nodo *Add constants true* que inyecta la variable booleana (*is_final* = Y). Si es falso, viaja por la rama inferior a *Add constants false* (*is_final* = N). Ambas ramas se reconcilian posteriormente en un nodo *Dummy* integrador.
5. **Carga Híbrida (*Combination lookup + Update*):** El pipeline implementa un patrón avanzado de persistencia en Hop. Primero, el nodo *Combination lookup/update* consolida las claves (*curso_academico* y *evaluacion*) e inserta la fila generando automáticamente la clave subrogada *id_tiempo*. Como este paso nativo no admite insertar atributos no-clave simultáneamente, el flujo encadena de forma síncrona un paso *Update*. Este busca el *id_tiempo* recién instanciado y le hace un UPDATE para persistir el metadato booleano *is_final* en la base de datos de destino.

7.2.5 Dimensión de asignatura

El modelo de asignaturas en el sistema origen transaccional presenta una arquitectura relacional altamente normalizada (en "Copo de Nieve"). Para modelarlo en el Data Warehouse bajo un esquema de Estrella óptimo para lectura, este pipeline colapsa jerarquías relacionales en una única fila dimensional (ver Figura 7.6).

1. **Ingesta y Búsquedas en Cascada (*DBLookup*):** Hop extrae *raw.asignatura* y, mediante tres nodos *DBLookup* encadenados, recorre las claves foráneas hacia arriba. Primero recupera el *num_optativas* desde la tabla *raw.curso*; después, obtiene la *especialidad*

Figura 7.5: Flujo del pipeline *dim_tiempo.hpl*.

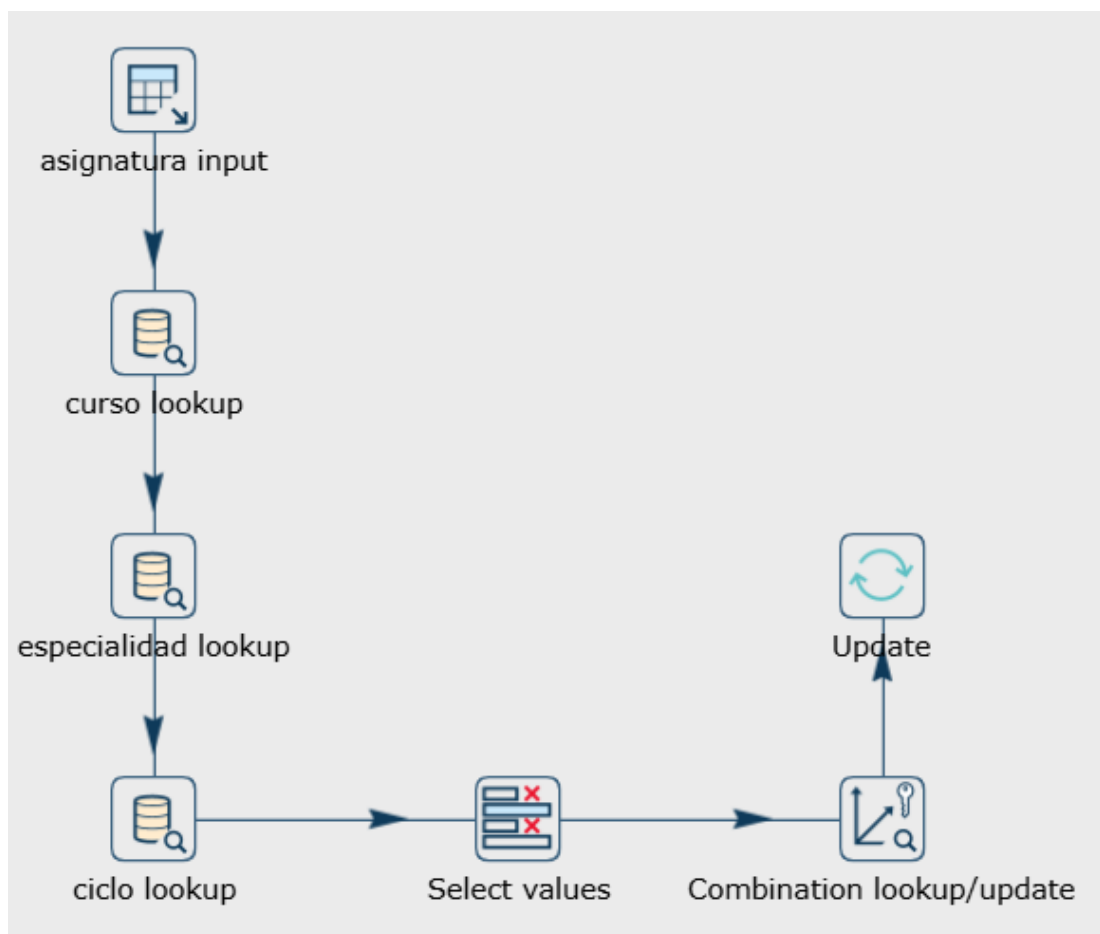
y su ciclo asociado desde *raw.especialidad*; y finalmente, extrae el marco normativo (*real_decreto*, *decreto_autonomico*) desde *raw.ciclo_educativo*.

2. **Consolidación (*SelectValues* y *Combination lookup*):** Una vez que todas las ramas jerárquicas han sido aplanadas en un único vector, se purgan las claves intermedias y se genera el registro en la dimensión destino con su *id_asignatura* correspondiente.

7.2.6 Dimensión de curso

Esta dimensión no solo almacena los metadatos del curso y ciclo, sino que alberga una regla de negocio fundamental: determinar la edad teórica esperada para un alumno en cada nivel, lo cual permitirá posteriormente a la tabla de hechos calcular el desfase de edad (indicador clave de repetición y riesgo académico, ver Figura 7.7).

1. **Mapeo de Nivel Educativo (*ValueMapper*):** Tras aplanar el curso, la especialidad y el ciclo educativo, un nodo inyecta paramétricamente la edad teórica de acceso al ciclo mediante pares clave-valor (ej. "PRI" → 5 años, "ESO" → 11 años, "BAC" → 15 años).
2. **Cálculo Algebraico de Edad Ideal (*Calculator*):** Un nodo matemático suma dinámicamente el número ordinal del curso a la edad base del ciclo. Así, un alumno en 3º

Figura 7.6: Flujo del pipeline *dim_asignatura.hpl*.

de la ESO (ciclo base 11) tiene una *edad_ideal* de 14 años. Este valor precalculado se consolida permanentemente en *dim_curso*.

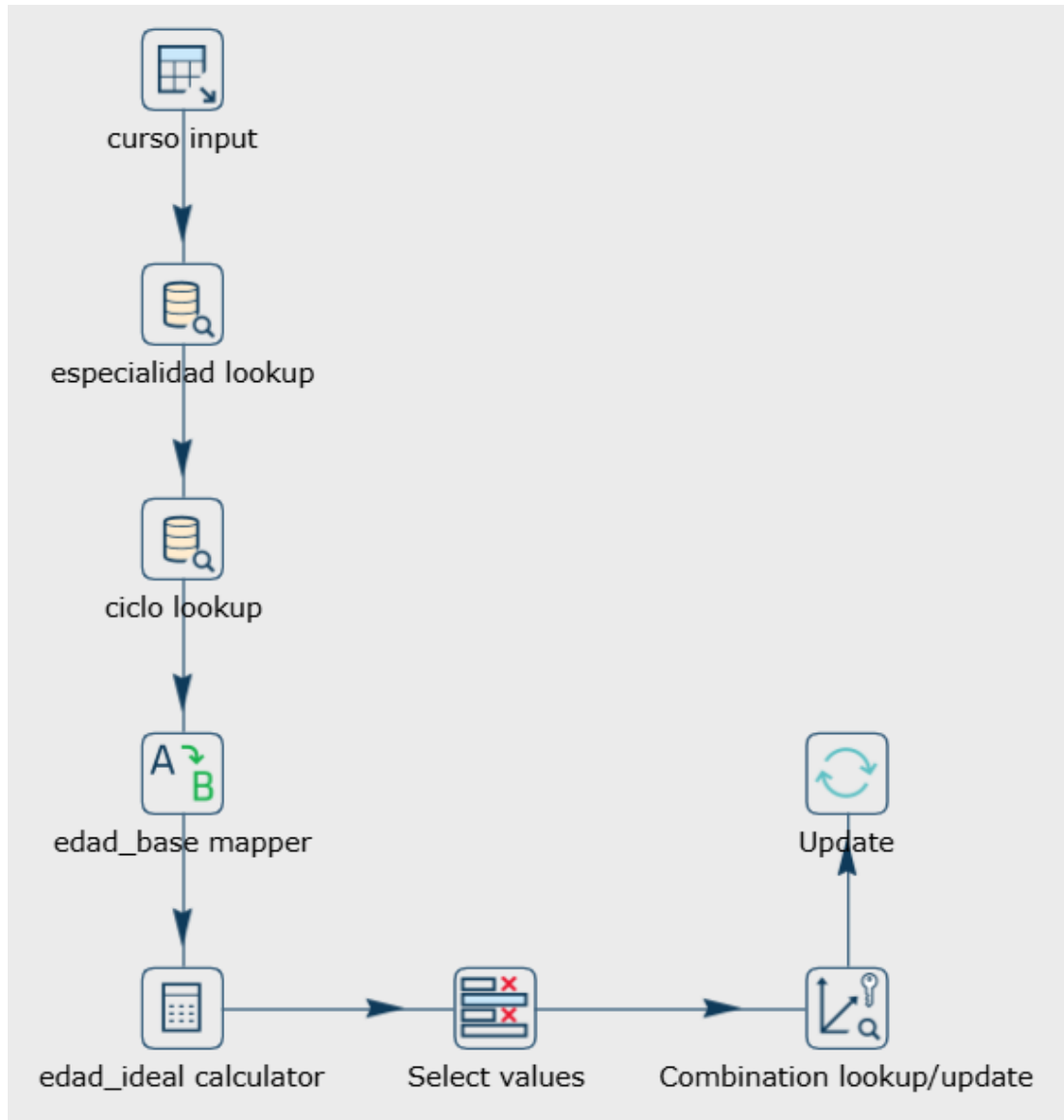
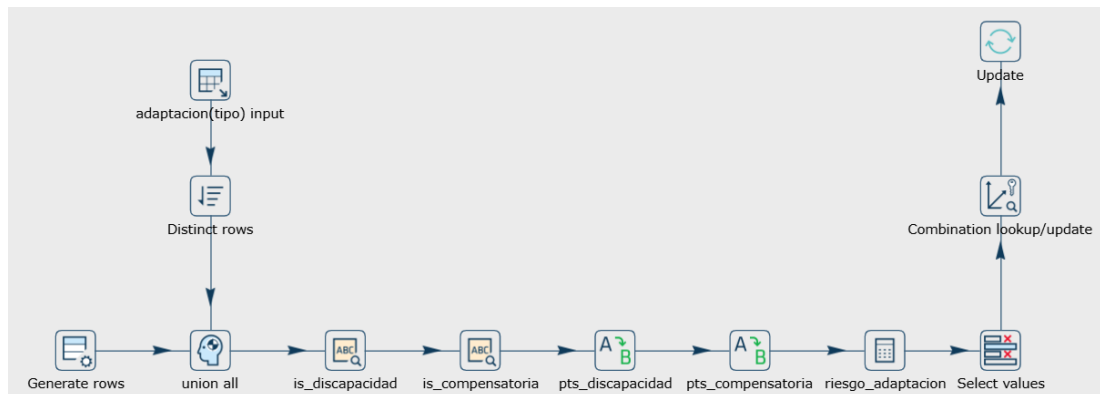


Figura 7.7: Flujo del pipeline *dim_curso.hpl*.

7.2.7 Dimensión de adaptación

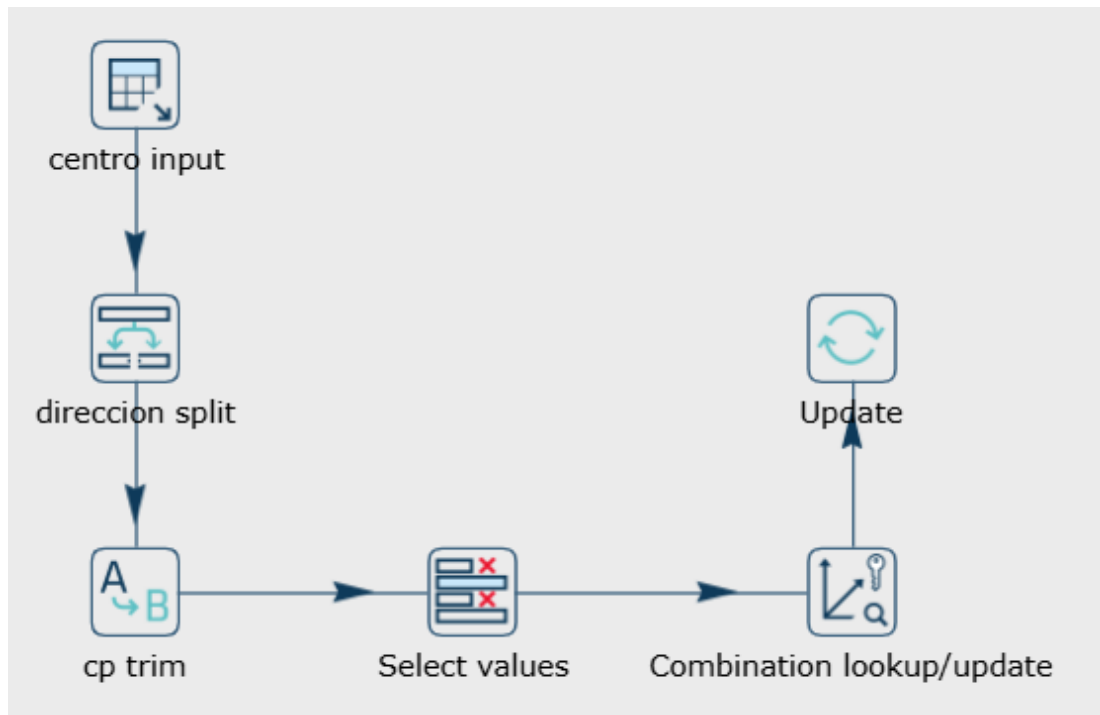
El registro transaccional de adaptaciones curriculares en los centros suele ser un campo de texto libre con la patología o motivo de la adaptación, dificultando su explotación analítica. Este pipeline actúa como un motor de extracción de reglas clínicas (ver Figura 7.8).

1. **Limpieza de Dominio (*SortRows* y *RowGenerator*):** Dado que la mayoría del alumnado no presenta adaptaciones, el flujo genera sintéticamente una fila "Ninguna" para asegurar la integridad referencial en el modelo de estrella, y posteriormente extrae la lista de tipos únicos registrados en el histórico.
2. **Clasificación Clínica por Expresiones Regulares (*RegexEval*):** Dos nodos paralelos aplican potentes patrones Regex sobre el texto crudo:
 - ***is_discapacidad*:** Busca que contenga (TND, Discalculia, Dislexia, Motórica, Visual, Asma, Mutismo, TDAH, TEA) para identificar discapacidades fisiológicas o neuropsicológicas.
 - ***is_compensatoria*:** Rastrea que incluya (Absentismo, Ludopatía, Acogimiento, Convivencia, Ciberacoso, Conducta, Intercultural, Social, Inestabilidad, Inmersión, Lingüística) para aislar intervenciones estrictamente sociológicas.
3. **Ponderación de Severidad (*ValueMapper* y *Calculator*):** Cada tipología desencadena un peso distinto (10 puntos para las fisiológicas, 5 puntos para las sociológicas). Un nodo matemático suma los puntos creando la métrica unificada *riesgo_adaptacion*.

Figura 7.8: Flujo del pipeline *dim_adaptacion.hpl*.

7.2.8 Dimensión de centro

El pipeline del centro educativo, aunque estructuralmente sencillo al procesar un catálogo estático y reducido de registros, encapsula lógicas de limpieza para la geolocalización analítica. El campo crudo *direccion* se somete a un nodo *FieldSplitter* que particiona los componentes por su coma delimitadora (en las columnas Calle, CP, y Ciudad), se limpia iterativamente la cadena del código postal eliminando el prefijo literario "CP " (*ReplaceString*), y se fuerza su formateo en base de datos al tipo nativo Integer en un paso *SelectValues* previo a la generación de la clave autoincremental *id_centro* (ver Figura 7.9).

Figura 7.9: Flujo del pipeline *dim_centro.hpl*.

7.2.9 Tabla de hechos de calificaciones

Este es el pipeline central del sistema. Su objetivo es poblar la tabla de hechos atómica *fact_calificaciones*, cruzando los eventos transaccionales con todas las dimensiones construidas previamente y calculando métricas volátiles en el momento del evento (ver Figuras 7.10 y 7.11).

1. **Ingesta y Cruce Base (*Table Input*, *SortRows*, *DBLookup*):** El flujo arranca extrayendo las calificaciones desde *raw.linea_expediente*, ordenándolas en memoria, y utilizando un nodo *DBLookup* ("matricula lookup") contra *raw.matricula* para heredar metadatos vitales como la *fecha* formal de matriculación, el *curso_academico* y el *cod_centro*.
2. **Resolución de Claves Estáticas (*DBLookup* en Cascada):** A continuación, se encadenan cuatro nodos idénticos de tipo *DBLookup* que interrogan a las dimensiones estáticas (*dim_asignatura*, *dim_centro*, *dim_curso* y *dim_tiempo*) cotejando las claves naturales para extraer sus identificadores subrogados (*id_asignatura*, etc.). Críticamente, del nodo "*dim_curso lookup*" no solo se extrae la clave, sino que se arrastra la variable precalculada *edad_ideal*.
3. **Gestión de Nulos en Adaptaciones (*StreamLookup* e *IfNull*):** Para resolver la dimensión de adaptaciones curriculares, el flujo lee asincrónicamente las tablas *raw* de

adaptaciones, hace un *StreamLookup* en memoria, e implementa un nodo *IfNull* programado para inyectar la cadena "Ninguna" si el alumno no presenta patologías registradas. Un *DBLookup* posterior extrae su *id_adaptacion*.

4. **Resolución Temporal SCD2 (*Dimension Lookup*):** Para enlazar la dimensión del estudiante, se emplea un potente nodo *Dimension Lookup* pasándole la *fecha* de matrícula extraída en el primer paso. Hop coteja automáticamente esta fecha contra los atributos *date_from* y *date_to* de la dimensión, devolviendo el *id_estudiante* que encapsula la realidad familiar que tenía el alumno **en el momento exacto de matricularse**.
5. **Cálculo Algebraico de Retraso Académico (*StringCut* y *Calculator*):** Un nodo *StringCut* ("*año_curso cut*") extrae los 4 primeros dígitos de la cadena *curso_academico* (ej. "2023" de "2023-2024"), y un paso *SelectValues* lo formatea a numérico. Posteriormente, tres nodos *Calculator* entran en juego: extraen el año de la fecha de nacimiento con *YEAR_OF_DATE*, restan ambos años para obtener la *edad_academica* del alumno, y le restan la *edad_ideal* arrastrada de la dimensión curso para obtener matemáticamente la métrica *desfase_edad*.
6. **Recálculo Demográfico Histórico (*DBJoin* y *Combination Lookup*):** El paso más exigente. Un nodo *DBJoin* interroga la tabla de encuestas inyectando la restricción SQL *fecha <= ?* (usando la fecha de la matrícula) para rescatar la encuesta vigente en aquel momento. A partir de sus columnas crudas, una decena de nodos recalculan las unidades de consumo (*Formula*), la renta por consumo (*Calculator*), categorizan los ingresos (*NumberRange*), discretizan el nivel de estudios (*ValueMapper* y *Formula*) y suman todos los puntos para inferir el *riesgo_socioeconomico*. Finalmente, el nodo avanzado *CombinationLookup* mapea estos atributos contra *dim_demografia_familiar* para asignar o crear el *id_demografia_familiar*.
7. **Carga Incremental tipo Upsert (*Insert / update*):** Superada la lógica de transformación, el pipeline utiliza un nodo transaccional *Insert / update* para volcar los datos en el esquema destino. Este componente evalúa en tiempo real si la combinación de claves ya existe en la tabla de hechos. Si existe, ejecuta una actualización (*Update*) sobre las métricas; si es un registro inédito, realiza una inserción limpia (*Insert*). Este mecanismo asegura la persistencia del historial acumulado y es el pilar que da soporte a la arquitectura *Change Data Capture (CDC)* de la solución.

7.2.10 Tabla de hechos de rendimiento por evaluación

El último proceso del ecosistema ETL actúa como el motor predictivo del proyecto, consolidando las calificaciones para calcular el riesgo final de abandono escolar. Para dotar al sis-

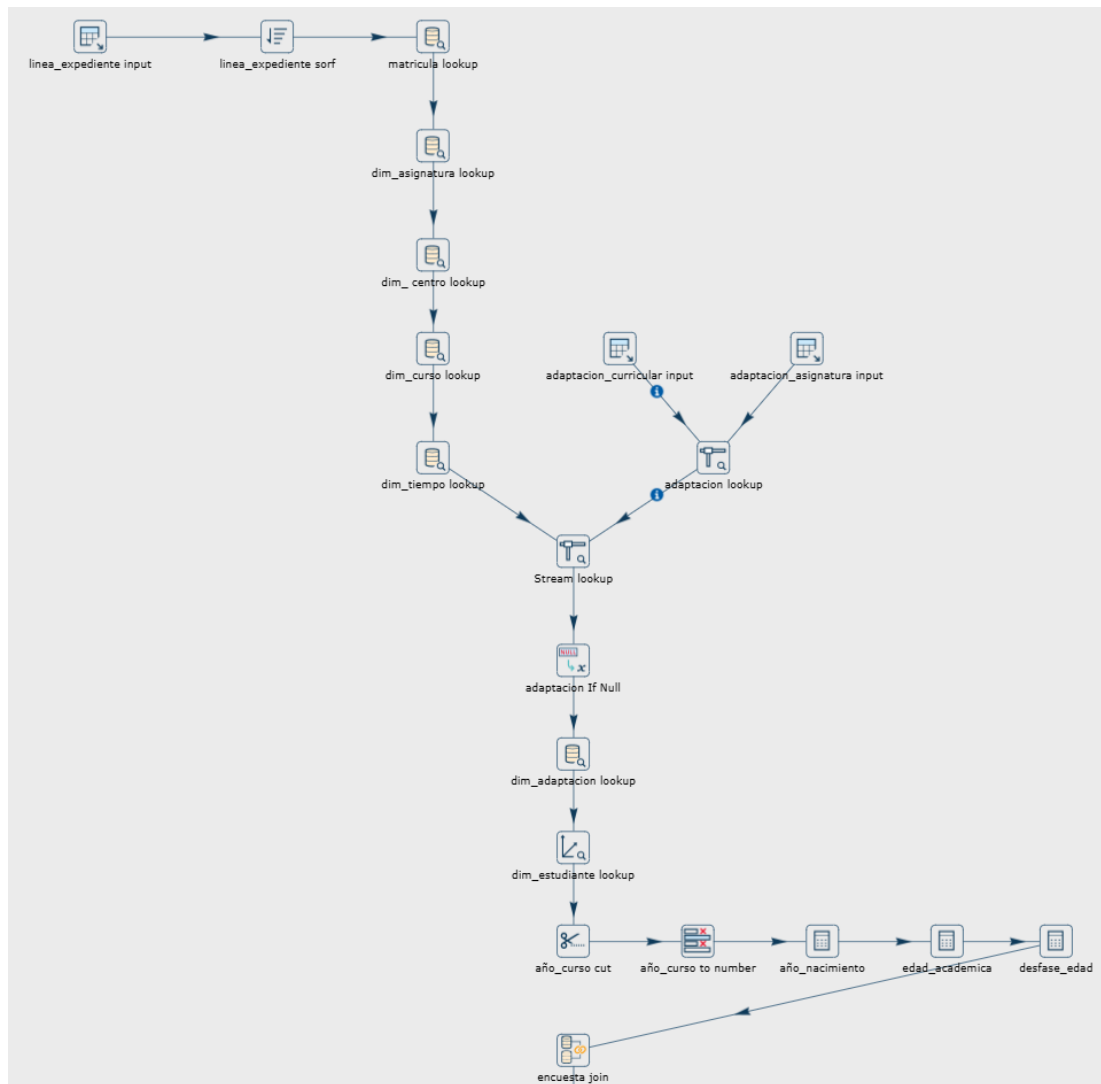
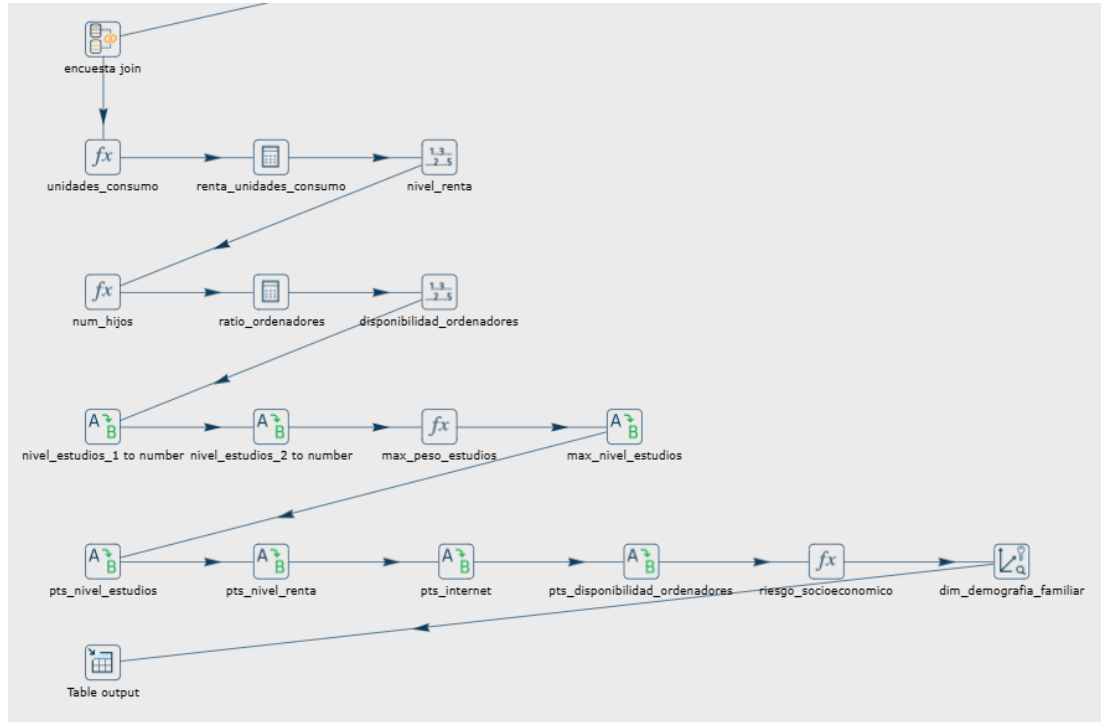


Figura 7.10: Flujo del pipeline *fact_calificaciones.hpl* (Parte 1).

Figura 7.11: Flujo del pipeline *fact_calificaciones.hpl* (Parte 2).

tema de la capacidad de procesar recargas históricas completas y, al mismo tiempo, soportar un modo de ejecución por evaluación de alta eficiencia, este cálculo se ha dividido en dos pipelines paralelos: *fact_rendimiento_anual_full.hpl* y *fact_rendimiento_anual_incremental.hpl*.

Lógica de consolidación masiva (Pipeline Full)

Este pipeline (ver Figuras 7.12 y 7.13) abandona la clásica consolidación estática anual y adopta una arquitectura de "Suma Acumulada" manteniendo el grano de la tabla a nivel de Evaluación ($id_estudiante \times id_tiempo$).

1. **Ingesta y Tipificación Analítica (Table Input, DBLookup y Formula):** El flujo ingiere *dwh.fact_calificaciones* y cruza mediante *DBLookup* con todas las dimensiones clave (*dim_tiempo*, *dim_curso*, *dim_demografia_familiar*, *dim_adaptacion*). Tras esto, un nodo *Formula* bautizado como "flags analíticos" perfila las casuísticas de cada nota. Es aquí donde se detecta de forma temprana si el registro pertenece al ciclo crítico de 1º y 2º de Primaria (*is_prim_1_2*), y se despliegan vectores binarios para segmentar si un suspenso es de evaluación final o parcial, y si pertenece a dicho ciclo crítico o al resto.
2. **Rastreo Histórico Acumulado (Flujo Principal):** Tras una ordenación cronológica estricta ($id_estudiante \times curso_academico \times orden_evaluacion$), un nodo *Group By*

agrupa por estudiante activando la directiva *"Include all rows"*. Implementando funciones *Cumulative sum* sobre los vectores previamente generados, el sistema emula una *Window Function*. Esto arrastra en el registro actual todo el bagaje histórico de suspensos segmentados (parciales/finales y primaria/resto).

3. **Motor Detección de Repetidores (Sub-rama Paralela):** El flujo se bifurca hacia una rama auxiliar especializada en trazar las repeticiones. Puesto que los parciales duplican los cursos, un nodo *Group By* colapsa la granularidad a un solo registro por año. Posteriormente, un nodo *Analytic Query* (LAG 1) lee el *curso_anterior* del alumno. Un nodo *Formula* detecta matemáticamente la repetición y separa mediante *flags* si esta se produce en 1º/2º de Primaria o en cursos superiores. Finalmente, un segundo *Group By* con sumatoria acumulativa calcula el cómputo histórico de repeticiones del alumno (*num_repetir_1_2_pri* y *num_repetir_resto*).
4. **Fusión y Reencuentro (*StreamLookup*):** Tras calcular la historia de repeticiones de la sub-rama, se hace un cruce (Left Join en memoria) mediante el nodo *StreamLookup*. Este vuelve a inyectar al flujo principal transaccional toda la mochila de repeticiones basándose en la clave *id_estudiante × curso_academico*.
5. **Ecuación Maestra de Riesgo (*Formula*):** Un potente nodo de fórmulas se encarga de convertir todas las acumulaciones en puntuaciones predictivas (Scoring). El **Riesgo Académico** se calcula ponderando de manera asimétrica los factores: suspensos finales en 1º/2º Primaria (20 pts), suspensos parciales (10 pts) y penalizando drásticamente la repetición en este ciclo temprano (40 pts), frente a las métricas del resto de cursos que tienen un peso menor. A la par, evalúa el riesgo base familiar creando una puntuación escalonada para el **Riesgo Socioeconómico** (10 pts si supera el umbral severo, 5 si es riesgo moderado).
6. **Cálculo del Indicador de Abandono (*Calculator*):** Como culminación, un nodo sumatorio simple (*ADD3*) consolida matemáticamente el *riesgo_academico*, el *riesgo_adaptacion* y los puntos socioeconómicos. Esto arroja el indicador definitivo *riesgo_abandono*, que finalmente es almacenado en la tabla *dwh.fact_rendimiento_anual* a través de un nodo masivo *Table Output*.

Adaptaciones del pipeline Incremental

El principal problema arquitectónico del flujo *Full* es que, para calcular la penalización por fracasos previos, necesita volcar en la memoria RAM el historial completo de calificaciones de toda la comunidad autónoma. En una carga inicial esto es necesario, pero en una recarga periódica de evaluación resulta insostenible por el enorme consumo de recursos y tiempo. Para re-

solver esta ineficiencia matemática, se diseñó la variante *fact_rendimiento_anual_incremental.hpl* (ver Figura 7.14), que aplica las siguientes soluciones:

1. **Ingesta Parametrizada y Recálculo del Curso:** El nodo *Table Input* inyecta la variable dinámica $\{CURSO_ACTUAL\}$, leyendo exclusivamente las calificaciones del ciclo lectivo en vigor. Se opta por recalcular el curso actual completo (en lugar de inyectar solo una única evaluación) para poder procesar y solventar automáticamente cualquier rectificación en las notas producida antes del cierre de actas, manteniendo un volumen de datos mínimo comparado con la carga histórica total.
2. **Recuperación del Historial (*Database Join*):** Al no disponer del bagaje histórico en RAM, el nodo *Group By* (Suma Acumulada) se vuelve inoperante a nivel interanual. Para solucionarlo, un nodo *Database Join* interroga directamente a la tabla destino (*fact_rendimiento_anual*) filtrando por el año anterior del alumno (ver detalle en la Figura 7.15). El objetivo clave es recuperar su *riesgo_academico* ya consolidado. Dado que el modelo matemático del riesgo es puramente aditivo, el sistema calcula de forma aislada los puntos de los suspensos del año actual y simplemente los suma al riesgo acumulado extraído de la base de datos.
3. **Detección de Repetidores Dinámica:** La sub-rama analítica basada en LAG 1 se descarta al carecer del año previo en memoria. El flujo aprovecha el cruce anterior con la propia tabla *fact_rendimiento_anual* para consultar si el alumno arrastra repeticiones previas (*num_repetir_1_2_pri* y *num_repetir_resto*), integrando estas penalizaciones en la fórmula actual de manera directa.
4. **Carga no Destructiva (*Insert / update*):** El nodo final abandona el volcado masivo y destructivo (*Table Output*). Utiliza en su lugar el componente transaccional *Insert / update*, permitiendo inyectar los nuevos cálculos de riesgo en el Data Warehouse mientras preserva intacto todo el historial de los años anteriores.

7.3 Orquestación y workflows

Todos estos pipelines independientes (*.hpl*) son llamados y secuenciados por flujos de trabajo o *Workflows* (*.hwf*). Mientras que los pipelines manejan registros fila a fila, los workflows manejan la lógica de control a nivel de proceso.

La orquestación del proyecto sigue una arquitectura jerárquica encabezada por el flujo principal *main.hwf* (ver Figura 7.16), el cual delega la carga en dos sub-workflows de especialidad:

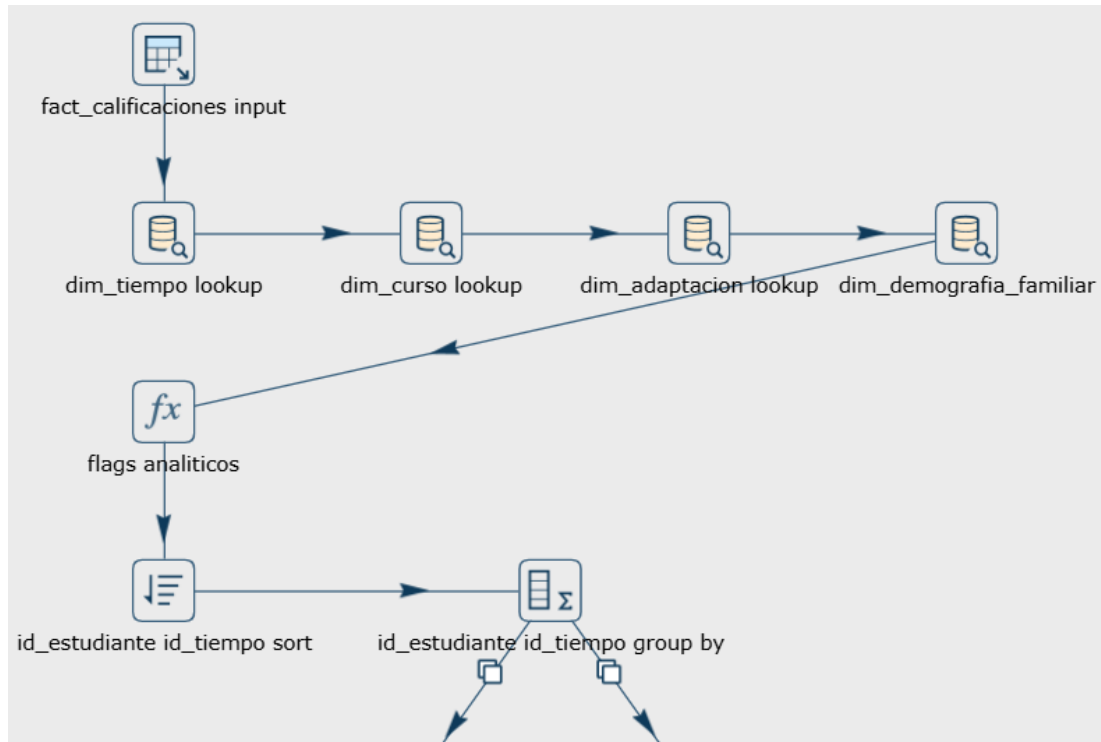
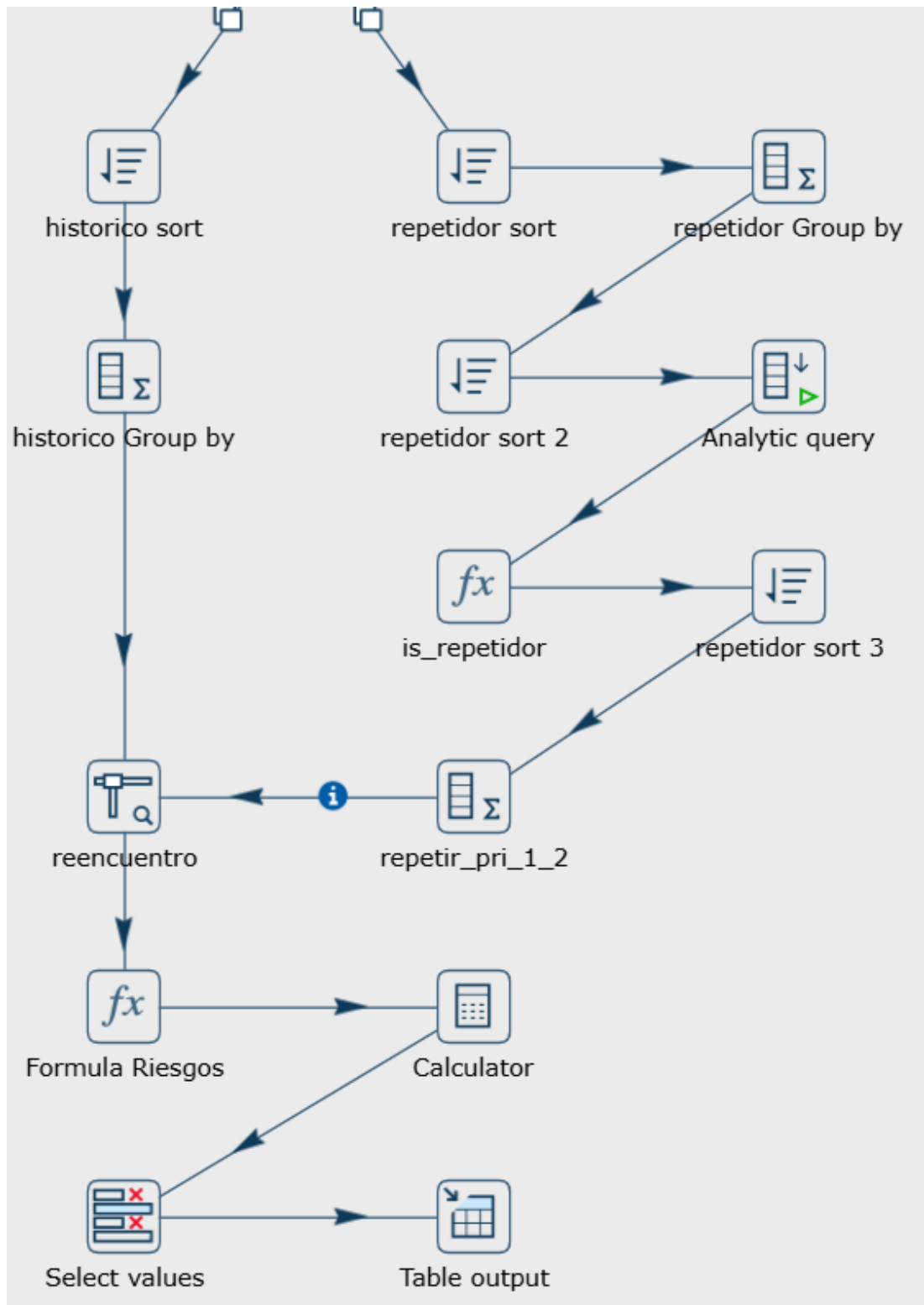


Figura 7.12: Flujo del pipeline *fact_rendimiento_anual_full.hpl* (Parte 1).

1. **dimensions.hwf**: Este flujo se encarga de la extracción inicial y la carga de dimensiones (ver Figura 7.17). Su ejecución comienza disparando el pipeline de extracción a *Staging*. Una vez los datos están en bruto en el esquema *raw*, el workflow lanza en **paralelo** la carga de todas las dimensiones (*dim_tiempo*, *dim_curso*, *dim_estudiante*, etc.) para maximizar el rendimiento. Es crítico que todas finalicen con éxito para garantizar la integridad referencial.
2. **facts.hwf**: Tras asegurar las dimensiones, *main.hwf* dispara este sub-workflow secuencial (ver Figura 7.18). Primero ejecuta el flujo atómico de calificaciones. Si finaliza con éxito, un nodo *Simple Evaluation* inspecciona el valor del parámetro $\{CURSO_ACTUAL\}$. Si la variable está vacía, el orquestador enruta la ejecución hacia el pipeline de rendimiento *Full* (carga masiva en memoria). Si contiene un curso específico, enruta dinámicamente hacia el pipeline *Incremental* (arquitectura híbrida base de datos-memoria), garantizando la máxima eficiencia operativa por evaluación.

7.3.1 Propagación de parámetros dinámicos

Una de las características más potentes de la orquestación implementada es la propagación jerárquica de variables. El proyecto evita el uso de valores "quemados" (*hardcoded*) en código,

Figura 7.13: Flujo del pipeline *fact_rendimiento_anual_full.hpl* (Parte 2).

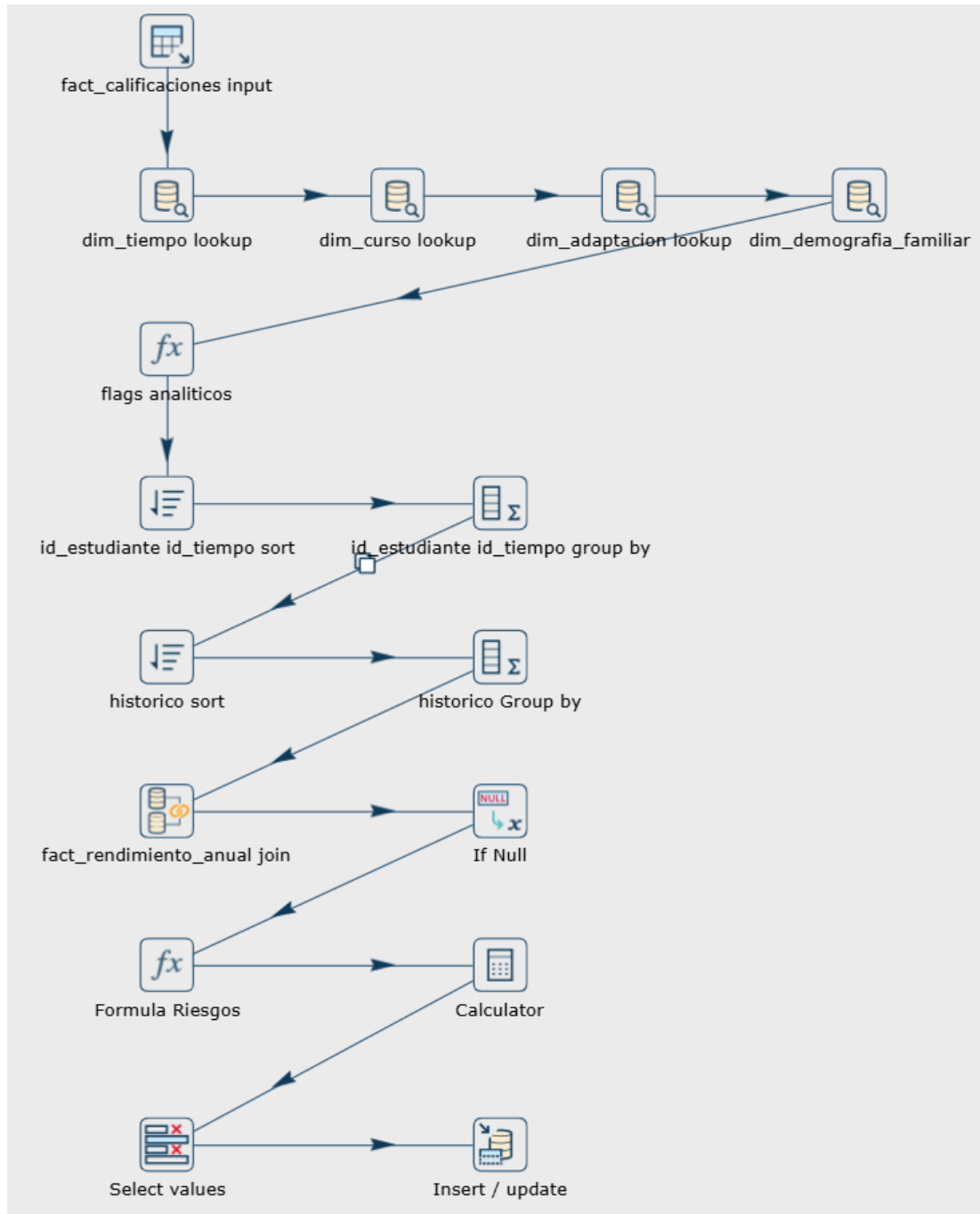


Figura 7.14: Flujo completo del pipeline *fact_rendimiento_anual_incremental.hpl*.

The screenshot shows the 'Database join' configuration window in Apache Hop. The window has a title bar with standard OS controls. Inside, the 'Transform name' field is set to 'fact_rendimiento_anual join'. The 'Connection' dropdown is set to 'postgres-dwh'. There are icons for saving, undo, redo, and other actions. The 'Enable cache' checkbox is unchecked. The 'Cache size in rows (0: cache everything)' field is set to 0. Below this is a large text area for the SQL query, containing a SELECT statement with joins and filters. At the bottom, there are settings for 'Number of rows to return' (0), 'Outer join' (checked), and 'Replace variables' (unchecked). A section titled 'The parameters to use:' contains a table with two parameters: 'id_estudiante' (Integer) and 'curso_academico' (String). At the very bottom are buttons for 'Help', 'OK', 'Get Fields', and 'Cancel'.

Transform name: fact_rendimiento_anual join

Connection: postgres-dwh

Enable cache: ☐

Cache size in rows (0: cache everything): 0

SQL

```
SELECT
riesgo_academico as hist_riesgo_academico,
id_curso as curso_anterior
FROM dwh.fact_rendimiento_anual f
JOIN dwh.dim_tiempo t ON f.id_tiempo = t.id_tiempo
WHERE f.id_estudiante = ?
AND t.curso_academico < ?
ORDER BY t.curso_academico DESC, t.orden_evaluacion DESC
LIMIT 1
```

Line 1 Column 1

Number of rows to return: 0

Outer join: ☒

Replace variables: ☐

The parameters to use:

#	Parameter fieldname	Parameter Type
1	id_estudiante	Integer
2	curso_academico	String

Buttons: ? Help, OK, Get Fields, Cancel

Figura 7.15: Nodo *Database Join* en *fact_rendimiento_anual_incremental.hpl*.

utilizando en su lugar el parámetro global *CURSO_ACTUAL* para gobernar el comportamiento de carga (Full o Incremental).

El ciclo de vida de este parámetro fluye en cascada o *top-down*:

1. **Inyección Externa:** El parámetro se define en el momento de la ejecución desde fuera del contenedor (por ejemplo, a través de variables de entorno en el *docker-compose* o argumentos del CLI de *Hop Run*).
2. **Recepción en el Workflow Principal:** El flujo *main.hwf* está configurado para declarar y capturar este valor en sus propiedades de ejecución, registrándolo en la memoria del orquestador.
3. **Transmisión a Sub-workflows:** A través de la configuración "*Pass parameter values to sub-workflow*", el *main* inyecta el curso actual hacia abajo, entregándolo como contexto válido a *dimensions.hwf* y *facts.hwf*.
4. **Consumo en los Pipelines:** Finalmente, los pipelines atómicos recogen la variable dinámica (invocada como *\$(CURSO_ACTUAL)*) y la inyectan directamente en el código SQL de los nodos *Table Input* para filtrar fechas, o la evalúan lógicamente en pasos de enrutamiento.

Este diseño asegura que todo el ecosistema de decenas de pipelines adapte su comportamiento instantáneamente a un nuevo año académico alterando un único valor externo de ejecución.

7.3.2 Manejo de errores multinivel (*error handling*)

La robustez del sistema recae en un diseño de gestión de excepciones presente en todas las capas. Tanto el *main.hwf* como los sub-workflows (*dimensions.hwf* y *facts.hwf*) implementan bloques de captura de errores.

Si cualquier pipeline de transformación individual reporta un fallo (por ejemplo, una clave foránea huérfana), el sub-workflow correspondiente detiene su rama de ejecución y se desvía hacia una acción *Write to log*. Allí registra la incidencia ("Uno de los pipelines de dimensión ha fallado") y lanza un *Abort workflow*. Este fallo se propaga jerárquicamente hacia arriba hasta el *main.hwf*, que también registra su propio error crítico y aborta la orquestación global. Esta topología previene cualquier corrupción o carga parcial en el Data Warehouse.

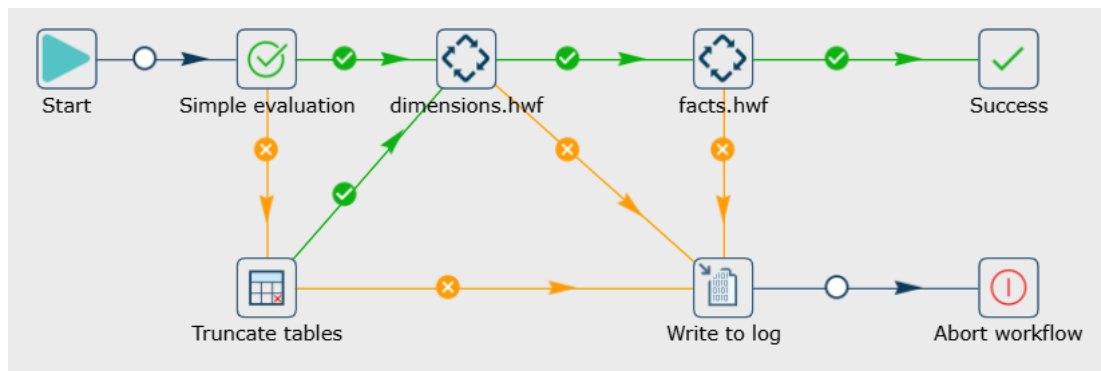


Figura 7.16: Workflow principal (*main.hwf*) y su delegación jerárquica en sub-flujos.

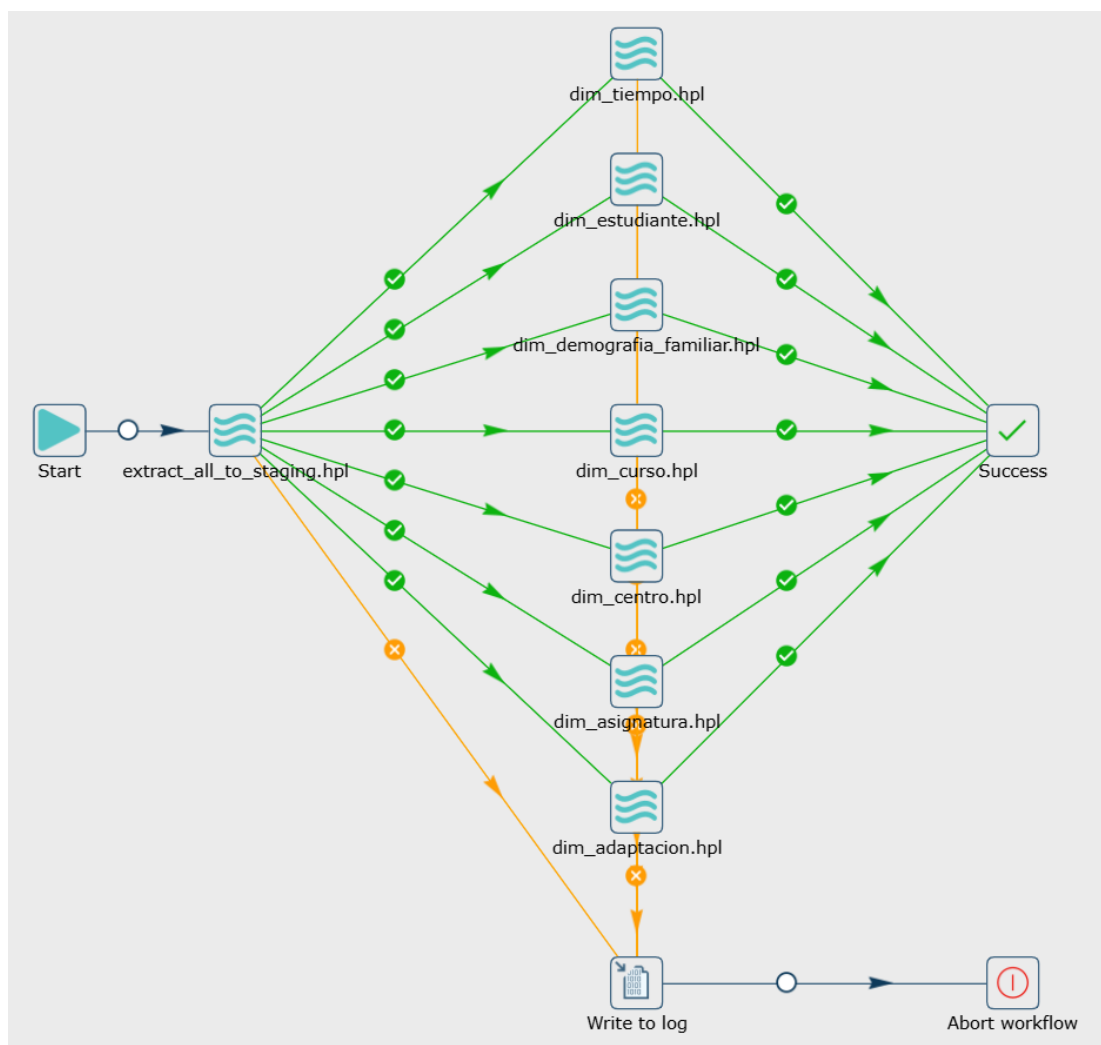


Figura 7.17: Sub-flujo de dimensiones (*dimensions.hwf*) ejecutado en paralelo.

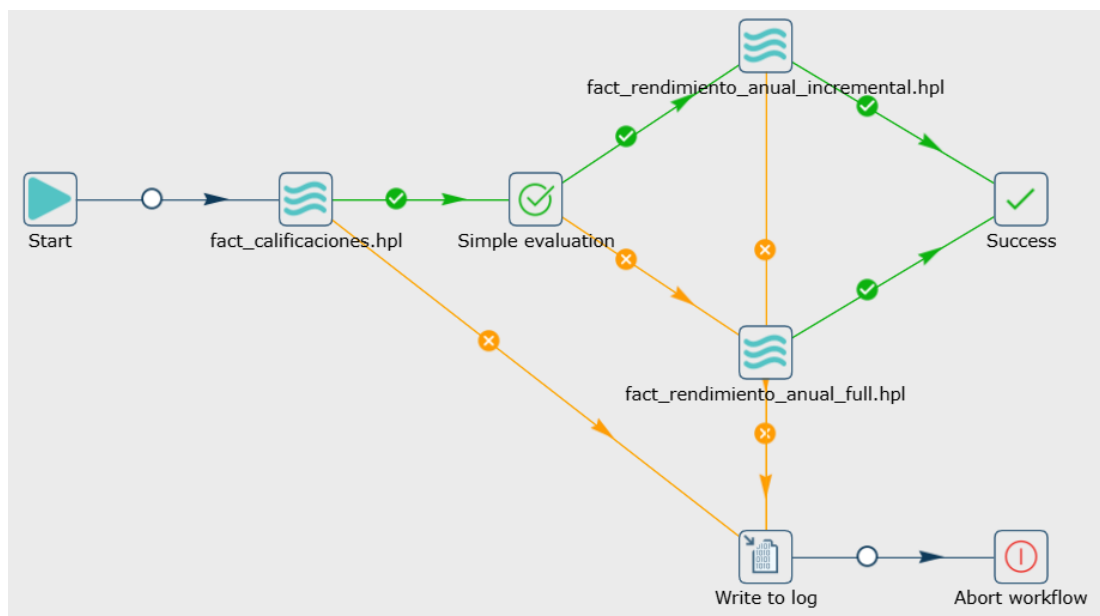


Figura 7.18: Sub-flujo de hechos (*facts.hwf*) ejecutado de forma secuencial.

Desarrollo del dashboard analítico

TRAS la consolidación de los datos en el Data Warehouse, la capa de presentación del proyecto consiste en una aplicación web que consume el Datamart de forma segura y devuelve agregaciones analíticas en tiempo de consulta. A diferencia de la capa ETL —cuyo único escritor es Apache Hop—, el dashboard no modifica las tablas del esquema *dwh*; su función es exclusivamente de lectura, agregación y exposición mediante una API REST.

Para cumplir con los requisitos de modularidad descritos en el Capítulo 5, la solución se ha desplegado en tres contenedores Docker independientes, interconectados por la red virtual *tfg-network*:

1. **Backend (api):** Servidor FastAPI en Python 3.12 que ejecuta las consultas SQLAlchemy sobre PostgreSQL, gestiona la autenticación JWT y expone los endpoints bajo el prefijo `/api/v1`.
2. **Frontend (frontend):** Aplicación React 18 empaquetada con Vite 6 y servida en producción mediante el paquete *serve* sobre el puerto interno 80.
3. **Proxy inverso (nginx):** Único punto de entrada hacia el exterior. Termina TLS, comprime respuestas y enruta el tráfico hacia los contenedores internos según la ruta solicitada.

La Figura 8.1 ilustra la topología completa de despliegue, mostrando cómo el tráfico HTTPS procedente del navegador atraviesa el proxy inverso y se bifurca hacia los servicios internos según el prefijo de la ruta.

8.1 Backend RESTful (FastAPI)

El núcleo de la lógica de negocio reside en el directorio `/api` del repositorio. Se ha implementado con **FastAPI** [10] sobre **Uvicorn**, la validación automática de tipos mediante **Pydantic v2** y la generación nativa de documentación OpenAPI accesible en `/docs` y `/redoc`.

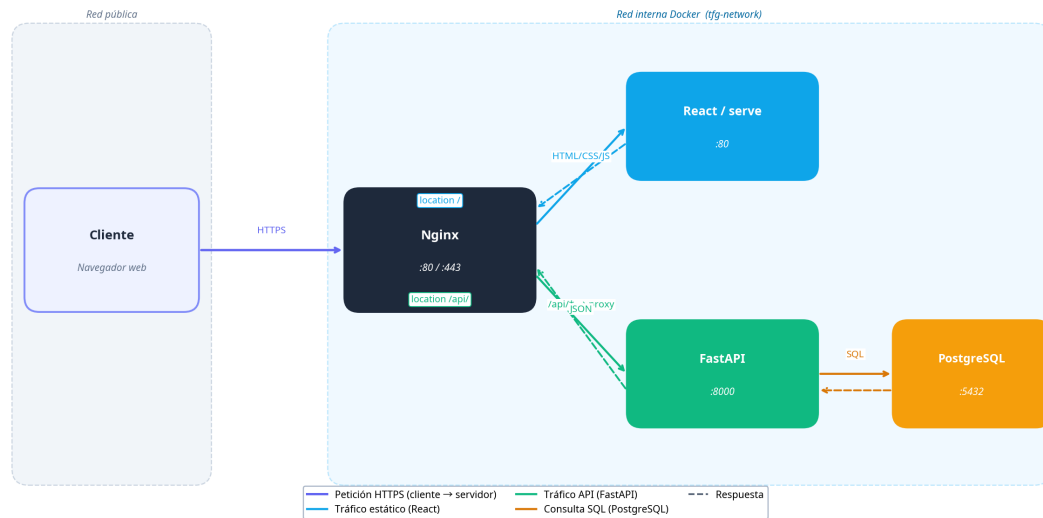


Figura 8.1: Diagrama de despliegue y enrutamiento del proxy inverso Nginx: separación de tráfico estático (React) y tráfico de API (FastAPI) dentro de la red Docker *tfg-network*.

8.1.1 Arranque y ciclo de vida del contenedor

El Dockerfile del backend construye una imagen a partir de `python:3.12-slim`, instala las dependencias de `requirements.txt` —incluyendo `libpq-dev` para el driver PostgreSQL— y define un comando de arranque en tres fases secuenciales:

1. **Migraciones Alembic:** `alembic upgrade head` crea o actualiza el esquema app en la base de datos postgres-dwh.
2. **Seeding:** `python -m app.seed` inserta el usuario administrador por defecto (ID fijo 1) si no existe.
3. **Servidor:** `uvicorn app.main:app --host 0.0.0.0 --port 8000` expone la API dentro de la red Docker.

El contenedor monta el directorio `./api` como volumen (`/app`), lo que permite recargar el código en desarrollo sin reconstruir la imagen. A diferencia del proxy Nginx, el servicio api **no publica puertos al host**: solo es accesible desde la red interna *tfg-network*, reforzando el aislamiento descrito en la arquitectura global.

8.1.2 Configuración centralizada (Pydantic Settings)

Toda la configuración sensible se externaliza mediante variables de entorno, leídas por la clase `Settings` en `app/core/config.py` con `pydantic-settings`. No existe ningún valor hardco-

deado de credenciales, hosts ni claves secretas en el código fuente. Las variables relevantes son:

- **Base de datos:** DB_DWH_HOST, DB_DWH_PORT, DB_DWH_DB, DB_DWH_USER, DB_DWH_PASS. Se ensamblan en la propiedad DATABASE_URI con el dialecto postgresql+psycopg2.
- **JWT:** JWT_SECRET_KEY, JWT_ALGORITHM (por defecto HS256).
- **CORS:** CORS_ORIGINS, parseable como lista JSON o cadena separada por comas.
- **Proyecto:** PROJECT_NAME y API_V1_STR (valor por defecto /api/v1).

8.1.3 Persistencia dual: esquemas app y dwh

El backend se conecta a la misma instancia PostgreSQL (postgres-dwh) que alberga el Datamart, pero opera sobre **dos esquemas lógicamente separados** mediante un patrón de ORM dual:

- **Esquema app (lectura/escritura):** Contiene la tabla users, gestionada por Alembic. La clase base en app/models/base.py fija metadata = MetaData(schema="app"). Es el único ámbito donde FastAPI realiza operaciones de inserción, actualización o borrado.
- **Esquema dwh (solo lectura):** Mapea las siete dimensiones y dos tablas de hechos del Datamart. La clase declarativa independiente DWHBase en app/models/datamart.py fija __table_args__ = {"schema": "dwh"} y no comparte metadata con base, de modo que Alembic no intente gestionar estas tablas.

8.1.4 Motor de base de datos y pool de conexiones

La configuración en app/core/database.py utiliza SQLAlchemy 2.0 en **modo síncrono** con psycopg2-binary. El *engine* se instancia con los siguientes parámetros de rendimiento:

- pool_size=10: conexiones persistentes en el pool.
- max_overflow=20: conexiones adicionales bajo picos de demanda.
- pool_pre_ping=True: verificación de conexión antes de cada uso.
- pool_recycle=1800: reciclaje de conexiones cada 30 minutos.

La dependencia get_db() genera una sesión SessionLocal por petición HTTP y garantiza su cierre en un bloque finally, evitando fugas de conexiones bajo carga concurrente.

8.1.5 Migraciones con Alembic

Alembic gestiona exclusivamente el esquema app. El fichero `alembic/env.py` filtra los esquemas mediante `include_name`, ejecuta `CREATE SCHEMA IF NOT EXISTS app` antes de aplicar migraciones y almacena la tabla de versiones en `app.alembic_version`. La migración inicial crea la tabla `app.users` con los campos: `id`, `email` (único, indexado), `hashed_password`, `role`, `created_at` y `updated_at`.

8.1.6 Autenticación y autorización

Flujo JWT

La autenticación sigue el esquema *Bearer Token*. La Figura 8.2 muestra la secuencia completa desde el login hasta la inyección automática del token en peticiones protegidas:

1. El cliente envía `POST /api/v1/auth/login` con `email` y `password`.
2. El backend busca al usuario en `app.users`, verifica la contraseña con `bcrypt` (vía `passlib`) y genera un JWT firmado con `HS256`.
3. Las peticiones subsiguientes incluyen la cabecera `Authorization: Bearer <token>`.
4. La dependencia `get_current_user` decodifica el token, valida firma y expiración, y recupera el usuario de la base de datos.

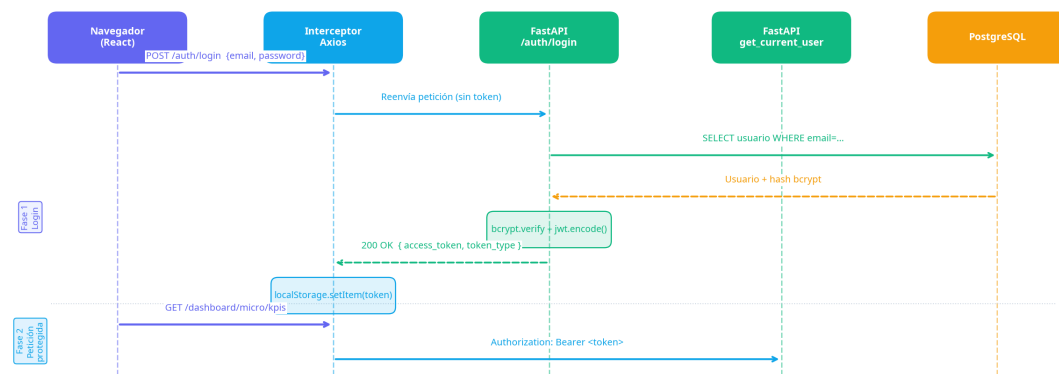


Figura 8.2: Diagrama de secuencia del flujo de autenticación JWT: fase de login (generación y almacenamiento del token) y fase de petición protegida (inyección del token por el interceptor Axios y validación por `get_current_user`).

Roles y permisos

Existen dos roles: admin y viewer (valor por defecto al crear usuarios). La dependencia `get_current_admin_user` restringe las operaciones de gestión de usuarios al rol admin. Además, el módulo `crud_user.py` implementa reglas de protección del superadministrador (usuario con `id=1`):

- Solo el superadmin puede crear, modificar o eliminar otros usuarios con rol admin.
- El superadmin no puede ser eliminado ni degradado a viewer.
- Otros administradores no pueden modificar la cuenta del superadmin.

El script `app/seed.py` crea el usuario inicial con email y contraseña configurable y rol admin, forzando `id=1`.

8.1.7 Arquitectura en capas del backend

El código del backend sigue una separación estricta de responsabilidades en cuatro capas, cuya organización se refleja en la Figura 8.3:

- **Routers** (`app/routers/v1/`): Definen los endpoints HTTP, validan parámetros de entrada con Query de FastAPI y delegan en la capa CRUD. Existen tres routers: `auth.py`, `dashboard.py` y `users.py`.
- **Schemas** (`app/schemas/`): Modelos Pydantic que definen la forma de las peticiones y respuestas JSON. Actúan como contrato tipado entre la API y el frontend.
- **CRUD** (`app/crud/`): Contiene la lógica SQL pesada. `crud_dashboard.py` concentra todas las agregaciones analíticas; `crud_user.py` gestiona el ciclo de vida de usuarios.
- **Models** (`app/models/`): Mapeo ORM de las tablas de ambos esquemas.

8.1.8 Diagrama de clases del backend

La Figura 8.3 resume la arquitectura orientada a capas del servidor FastAPI, distinguiendo tres niveles que se corresponden con los paquetes reales del repositorio:

- **Interfaces REST** (`app/routers/v1/`): Los tres `APIRouter` exponen los *endpoints* HTTP. Cada función de ruta valida los parámetros de entrada, inyecta las dependencias de autenticación (`get_current_user`) y delega en la capa inferior. Los modelos Pydantic de `app/schemas/` actúan como contrato de serialización JSON (peticiones y respuestas), generando automáticamente la documentación OpenAPI.

- **Objetos de negocio** (app/crud/ y app/core/security.py): Concentran la lógica analítica y las reglas de acceso. crud_dashboard.py implementa las agregaciones SQL, los umbrales de riesgo, la comparativa T0/T-1 y el índice de riesgo por centro; crud_user.py gestiona el ciclo de vida de usuarios y las restricciones del superadministrador. El módulo security.py encapsula el hashing bcrypt y la emisión/validación de JWT.
- **Acceso a datos** (app/models/ y app/core/database.py): SQLAlchemy mapea las tablas de los esquemas app (User) y dwh (dimensiones y hechos del Datamart). Un único engine y la factoría SessionLocal proveen sesiones transaccionales por petición mediante get_db().

El flujo de una petición autenticada al dashboard sigue la cadena: *Router* → *CRUD* → *Session/Model* → PostgreSQL. La respuesta se valida contra el *Schema* Pydantic correspondiente antes de devolverse al cliente React.

8.1.9 Motor de consultas analíticas (crud_dashboard.py)

Todas las consultas del dashboard parten de la función interna `_base_query()`, que realiza un JOIN de `fact_rendimiento_anual` con las seis dimensiones necesarias: `dim_estudiante`, `dim_centro`, `dim_curso`, `dim_tiempo`, `dim_adaptacion` y `dim_demografia_familiar`. La Figura 8.4 ilustra este modelo en estrella, detallando las claves primarias y foráneas que articulan cada JOIN. Los identificadores de las claves foráneas en la tabla de hechos se castean a Integer en los JOIN para compatibilizar tipos BigInteger con las claves subrogadas de las dimensiones.

Filtros dinámicos

La función `_apply_filters()` acepta cuatro parámetros opcionales como listas, permitiendo selección múltiple en el frontend mediante query params repetidos (ej. `?cod_centro=A&cod_centro=B`):

- `cod_centro`: filtra por `dim_centro.cod_centro`.
- `curso_academico`: filtra por `dim_tiempo.curso_academico`.
- `cod_ciclo`: filtra por `dim_curso.cod_ciclo`.
- `tipo_centro`: filtra por `dim_centro.tipo`.

Umbrales de riesgo dinámicos

La función `get_dynamic_risk_thresholds()` calcula los umbrales de clasificación bajo, medio y alto a partir del máximo histórico de `riesgo_abandono` en el Datamart: el umbral medio

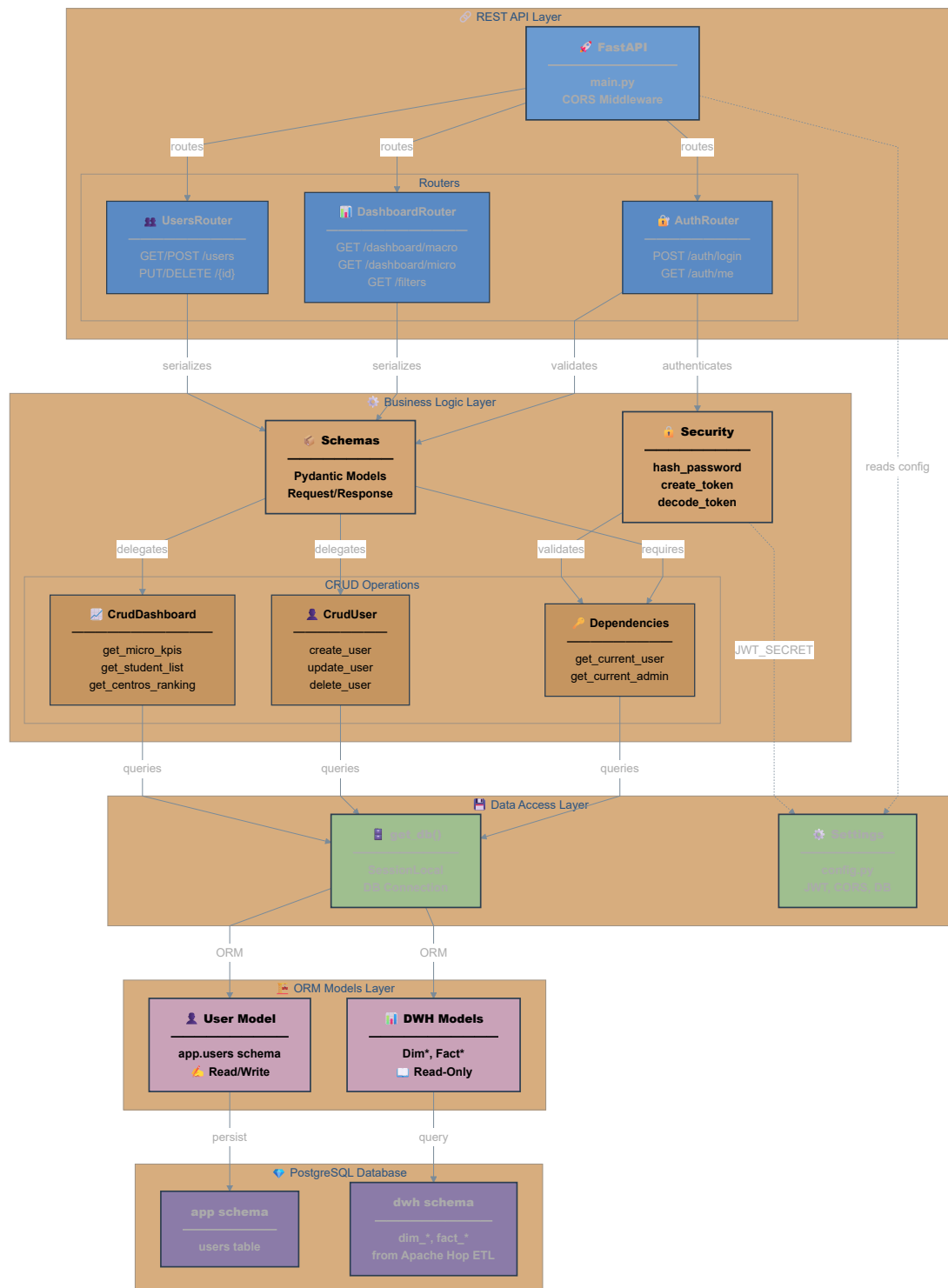


Figura 8.3: Diagrama de clases simplificado del backend FastAPI: interfaces REST, lógica de negocio y acceso a datos.

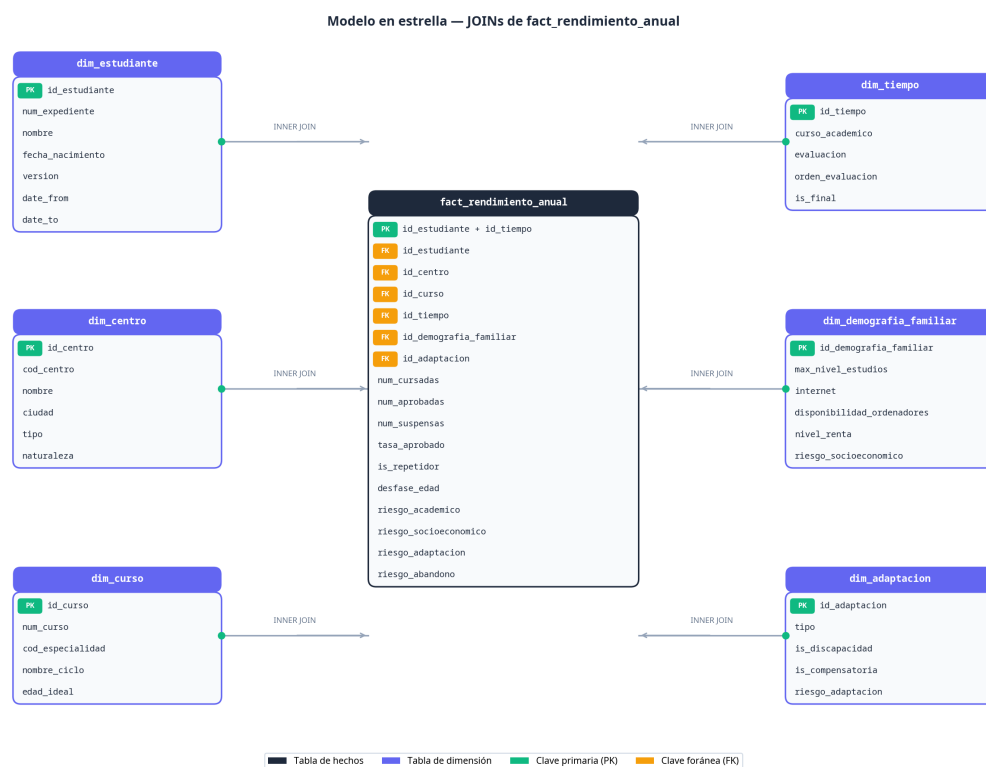


Figura 8.4: Modelo en estrella: JOINS entre fact_rendimiento_anual y las seis dimensiones del datamart utilizados como base de todas las consultas analíticas del backend.

corresponde al 25% del máximo y el alto al 50%. Esto evita umbrales fijos que quedarían obsoletos si el ETL recalcula las ponderaciones.

Para el riesgo socioeconómico se emplean constantes fijas definidas en el módulo: `RIESGO_SOCIO_ALTO_UMBRAL = 16` y `RIESGO_SOCIO_MEDIO_UMBRAL = 5`, alineadas con las reglas de negocio del Capítulo 6.

Comparativa interanual (T0 vs T-1)

Las funciones de KPIs (`get_micro_kpis`) y ranking de centros (`get_centros_ranking`) implementan una lógica de comparación temporal. Si el cliente no especifica curso académico, se toma el más reciente del Datamart. A partir de ese año (T0), la función auxiliar `get_a1()` calcula el curso anterior (T-1), soportando formatos 2023-2024, 23/24 o años simples. Los contadores se agrupan por año y se devuelven aplanados con sufijo `_a1` para el periodo anterior.

Índice de riesgo por centro

La función `get_centros_ranking()` agrega diez métricas de alerta por centro educativo (riesgo alto, repetidores, suspensos en primaria, brecha digital, etc.), calcula el porcentaje de alumnos afectados en cada una y determina la mediana de cada métrica entre todos los centros. El **índice de riesgo del centro** (`indice_riesgo_centro`) es el número de métricas cuyo porcentaje supera la mediana autonómica (escala 0–10). Los centros se ordenan de forma descendente por este índice.

Adicionalmente, el backend geocodifica cada centro mediante la librería `geopy` (geocodificador Nominatim de OpenStreetMap), usando la pareja código postal + ciudad. Las coordenadas se cachean en memoria a nivel de módulo (`_GEO_CACHE`) para evitar llamadas repetidas a la API externa.

8.1.10 Catálogo de endpoints

La Figura 8.5 recoge el inventario completo de endpoints expuestos por el router `dashboard.py`. Todos requieren autenticación JWT salvo `POST /auth/login`.

El endpoint `GET /dashboard/micro/students` admite, además de los cuatro filtros globales, parámetros de alerta booleanos (`alerta_brecha_digital`, `alerta_renta_baja`, `alerta_riesgo_alto`, etc.), ordenación configurable (`sort_by`, `sort_desc`) y paginación (`page`, `page_size`, máximo 100 registros por página).

La convención de rutas alinea el prefijo de cada endpoint con la vista del frontend que lo consume: `/macro/*` para el *MacroDashboard* y `/micro/*` para el *MicroDashboard*, más `/filters` como recurso compartido.

Método y ruta	Schema de respuesta	Descripción
POST /auth/login	TokenResponse	Autenticación por email/contraseña
GET /auth/me	UserResponse	Perfil del usuario autenticado
GET /dashboard/filters	FiltersResponse	Valores únicos para los filtros del sidebar
GET /dashboard/macro/kpis	MacroKPIsResponse	Centros por encima de la media en cada métrica
GET /dashboard/macro/centros	CentroListResponse	Ranking paginado de centros con geolocalización
GET /dashboard/macro/charts/trend	TrendMacroResponse	Tendencia histórica de riesgo medio
GET /dashboard/macro/charts/risk-by-type	RiskByTypeResponse	Riesgo medio por tipo de centro
GET /dashboard/micro/kpis	KPIsResponse	KPIs de alumnado con comparativa T0/T-1
GET /dashboard/micro/students	StudentListResponse	Listado paginado de alumnos con alertas
GET /dashboard/micro/charts/risk-distribution	RiskDistributionResponse	Distribución Bajo/Medio/Alto
GET /dashboard/micro/charts/risk-by-cycle	RiskByCycleResponse	Riesgo crítico por ciclo educativo
GET /dashboard/micro/charts/trend	TrendResponse	Evolución histórica (últimos 5 años)
GET /dashboard/micro/charts/failed-subjects	FailedSubjectsDistributionResponse	Distribución de suspensos por alumno
GET /dashboard/micro/charts/income-distribution	IncomeDistributionMicroResponse	Distribución del nivel de renta
GET /dashboard/micro/charts/correlation-income-failures	CorrelationIncomeFailuresResponse	Dispersión renta vs. suspensos
GET /dashboard/micro/charts/digital-gap	DigitalGapResponse	Brecha digital y tasa de aprobados
GET /dashboard/micro/charts/parent-education	ParentEducationResponse	Nivel de estudios de los progenitores
GET /dashboard/micro/charts/income-risk	IncomeRiskResponse	Renta y riesgo crítico de abandono
GET /users	List[UserResponse]	Listado de usuarios (solo admin)
POST /users	UserResponse	Crear usuario (solo admin)
PUT /users/{id}	UserResponse	Actualizar usuario (solo admin)
DELETE /users/{id}	UserResponse	Eliminar usuario (solo admin)

Autenticación
Filtros
Dashboard Macro
Dashboard Micro
Gestión de Usuarios

Figura 8.5: Catálogo de endpoints de la API del dashboard (/api/v1).

8.2 Frontend (React y Vite)

La interfaz del dashboard reside en el directorio `/frontend` y se ha desarrollado como una **Single Page Application (SPA)** con React 18 [11] y el bundler **Vite** 6. La elección de Vite frente a herramientas como Webpack se fundamenta en su arranque casi instantáneo en desarrollo (basado en ESM nativo) y en la generación de bundles de producción optimizados mediante Rollup.

8.2.1 Pipeline de construcción y despliegue

El Dockerfile del frontend implementa un **build multi-etapa**, cuyas fases se ilustran en la Figura 8.6:

1. **Etapla build:** Imagen `node:20-alpine`. Instala dependencias con `npm install` y ejecuta `npm run build`, generando los artefactos estáticos en `/app/dist`.
2. **Etapla de producción:** Imagen `node:20-alpine` limpia. Instala globalmente el paquete `serve` y ejecuta `serve -s dist -l 80`, sirviendo la SPA con *fallback* a `index.html` para las rutas del cliente.

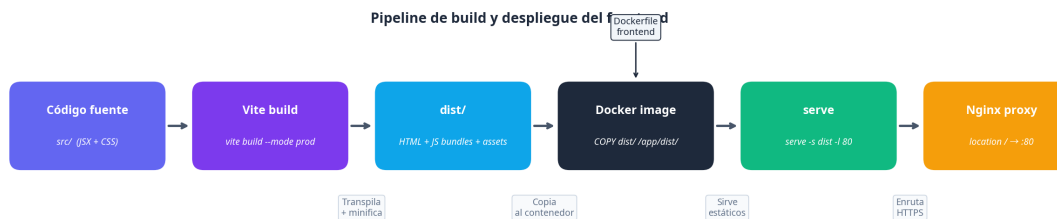


Figura 8.6: Pipeline de build y despliegue del frontend: desde el código fuente JSX hasta el servicio de ficheros estáticos a través del proxy Nginx.

Es importante señalar que el fichero `frontend/nginx.conf` existe en el repositorio como configuración alternativa de referencia, pero **no se utiliza** en el contenedor de producción actual, que delega el servicio de ficheros estáticos a `serve`.

8.2.2 Dependencias del proyecto

Las dependencias de producción declaradas en `package.json` son:

- **axios** (v1.7.9): Cliente HTTP con soporte de interceptores para la inyección automática del JWT.
- **react-router-dom** (v6.28.0): Enrutamiento del lado del cliente.
- **recharts** (v2.13.3): Librería declarativa de gráficos basada en D3.
- **leaflet + react-leaflet**: Mapa interactivo para la geolocalización de centros.
- **lucide-react**: Iconografía vectorial utilizada en tarjetas KPI y navegación.

8.2.3 Arquitectura del código fuente

El directorio `src/` se organiza en los siguientes módulos técnicos, cuya estructura se resume en la Figura 8.7:

- **services/api.js**: Instancia central de Axios con `baseUrl` configurable vía `VITE_API_URL` (por defecto `/api/v1`). Implementa un interceptor de peticiones que inyecta el JWT desde `localStorage` y un interceptor de respuestas que redirige a `/login` ante errores 401.
- **api/dashboard.js**: Capa única de acceso a la API del dashboard. Centraliza la serialización de filtros y expone una función por endpoint (`getMacroCentros`, `getMicroKpis`, etc.), evitando rutas duplicadas en los componentes.
- **context/**: Dos contextos React para estado global:
 - **AuthContext**: Gestiona token, usuario autenticado, login/logout y validación inicial del token contra `GET /auth/me`.
 - **DashboardContext**: Almacena el tema visual (`light/dark`), los filtros globales y el estado de alertas activas para *cross-filtering*.
- **pages/**: Tres vistas autenticadas: `MacroDashboard`, `MicroDashboard` y `UserManagement`, más `LoginPage`.
- **components/**: Componentes reutilizables organizados por dominio: `charts/` (gráficos Recharts), `dashboard/` (mapa y tarjetas de centro), `layout/` (sidebar y cabecera), `ui/` (tarjetas KPI, badges, tabla de alumnos) y `users/` (modal de gestión).

8.2.4 Enrutamiento y protección de rutas

El fichero `App.jsx` define la estructura de rutas con `react-router-dom`, cuya jerarquía de protección se corresponde con la separación de módulos mostrada en la Figura 8.7:

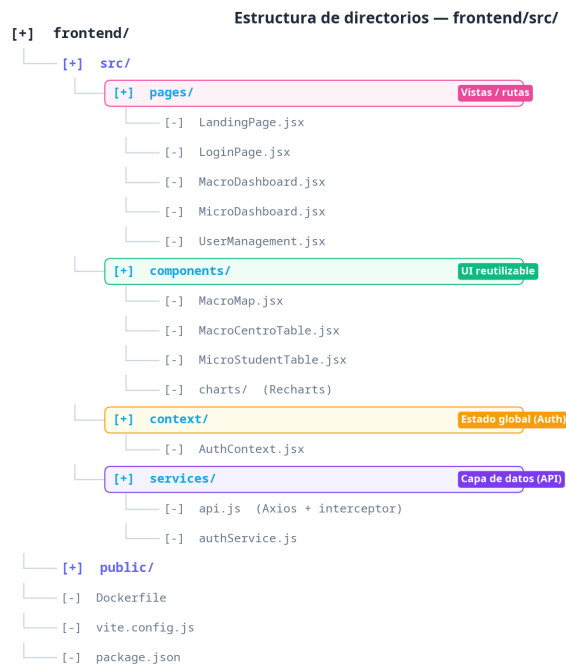


Figura 8.7: Estructura de directorios del frontend (src/): separación en pages/ (vistas), components/ (UI reutilizable), context/ (estado global) y services/ (capa de datos).

- `/login`: Página de autenticación. Redirige a `/macro` si ya existe sesión.
- `/` (protegida): Layout principal (`AppLayout`) con sidebar y área de contenido (`<Outlet />`). Requiere autenticación; si no hay token, redirige a `/login`.
 - `/macro`: Vista de análisis autonómico (ranking de centros, mapa, gráficos macro).
 - `/micro`: Vista de análisis granular (KPIs, gráficos detallados, tabla de alumnos).
 - `/users`: Gestión de usuarios (accesible según rol).

El árbol de componentes se envuelve en `ErrorBoundary` para capturar errores de renderizado sin derribar la aplicación completa.

8.2.5 Sistema de diseño (CSS nativo)

El frontend no utiliza frameworks de componentes externos (Bootstrap, Material UI, etc.). El sistema de diseño está implementado íntegramente en CSS puro mediante variables personalizadas en `index.css`, siguiendo una estética inspirada en el *Glassmorphism* de los sistemas operativos modernos. Estas variables se aplican de forma transversal a todos los componentes del directorio `components/` visible en la Figura 8.7:

- Variables de color semánticas: `-color-primary`, `-color-danger`, `-color-success`, `-color-warning`.
- Efectos de cristal: `-glass-blur (blur(24px))`, `-glass-border` y `-glass-shadow`.
- Soporte de tema oscuro mediante el atributo `[data-theme="dark"]` en el elemento `<html>`, alternado por `DashboardContext` y persistido en `localStorage`.
- Tipografía: familia *Inter* cargada desde Google Fonts.

8.2.6 Integración con la API

Cada componente de gráfico consume la API exclusivamente a través de `api/dashboard.js`, pasando los filtros globales desde `DashboardContext`. Este flujo de datos, desde el interceptor Axios hasta el endpoint del backend, queda representado en la Figura 8.2 para el caso de peticiones autenticadas. En desarrollo local, `vite.config.js` configura un proxy de `/api` hacia `http://api:8000`, evitando problemas de CORS durante el desarrollo con `npm run dev` (puerto 5173).

8.3 Proxy inverso y despliegue (Nginx)

El contenedor nginx constituye el **único punto de entrada** de la aplicación web hacia el exterior. Su configuración reside en `nginx/nginx.conf` y se despliega mediante una imagen `nginx:alpine` personalizada.

8.3.1 Terminación TLS y redirección HTTP

Nginx escucha en dos puertos, mapeados al host mediante las variables `NGINX_HTTP_PORT` y `NGINX_HTTPS_PORT` del fichero `.env`:

- **Puerto 80 (HTTP):** Redirige permanentemente (código 301) todas las peticiones a HTTPS, preservando el host y la ruta original.
- **Puerto 443 (HTTPS):** Servidor principal con certificados TLS montados desde el volumen `./nginx/ssl` (`nginx.crt` y `nginx.key`). Se habilitan los protocolos TLSv1.2 y TLSv1.3 con cifrados seguros.

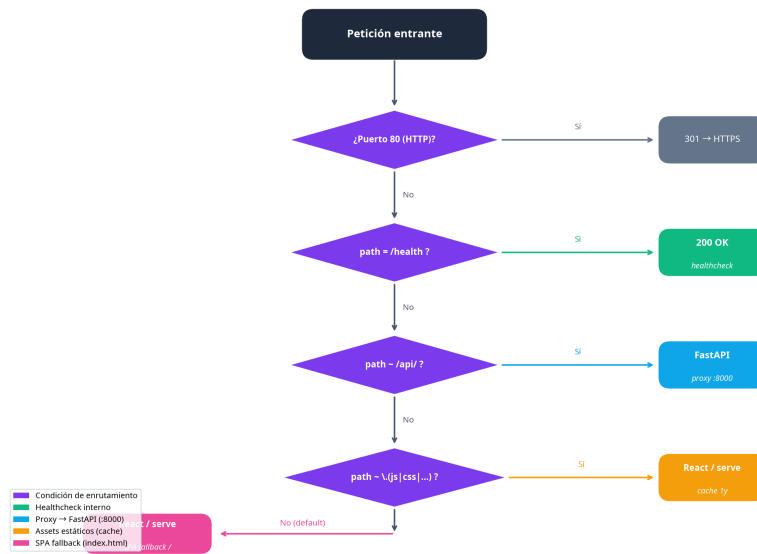
8.3.2 Reglas de enrutamiento

El fichero de configuración define dos *upstreams* internos de Docker:

- `frontend` → `frontend:80` (contenedor React/serve).
- `api` → `api:8000` (contenedor FastAPI/Uvicorn).

Las reglas de location implementan el siguiente comportamiento, cuyo flujo de decisión se detalla en la Figura ??:

1. `/health`: Endpoint de *health check* propio de Nginx. Devuelve 200 healthy sin acceder a los backends. Utilizado por el healthcheck de Docker Compose.
2. `/api/`: *Proxy pass* transparente al upstream `api`. Se reenvían las cabeceras X-Real-IP, X-Forwarded-For y X-Forwarded-Proto. Los *timeouts* de conexión, envío y lectura se fijan en 60 segundos para tolerar consultas analíticas pesadas.
3. `/ws/`: Soporte de *WebSocket* con cabeceras Upgrade y Connection: upgrade (preparado para futuras extensiones en tiempo real).
4. Ficheros estáticos (`.js`, `.css`, imágenes, fuentes): Se proxifican al contenedor `frontend` con cabeceras de caché de un año (Cache-Control: public, immutable).
5. `/` (resto de rutas): Se proxifican al contenedor `frontend`, que resuelve el *fallback* SPA hacia `index.html`.



captionDiagrama de flujo de decisión del proxy inverso Nginx: enrutamiento según el path de la petición hacia /health, /api/, assets estáticos o la SPA React.

8.3.3 Optimizaciones de rendimiento

Nginx aplica compresión gzip a tipos MIME relevantes (JSON, JavaScript, CSS, SVG) con nivel de compresión 6 y umbral mínimo de 256 bytes. El tamaño máximo del cuerpo de petición se limita a 20 MB (`client_max_body_size`). Estas optimizaciones complementan el flujo de enrutamiento descrito en la Figura ??.

8.3.4 Topología de red resultante

Desde la perspectiva de un cliente externo, la topología de acceso es la siguiente (véase también la Figura 8.1 al inicio del capítulo):

- El navegador se conecta exclusivamente a Nginx (HTTPS).
- Las peticiones a `/api/v1/*` son reenviadas internamente a FastAPI, que consulta PostgreSQL (`postgres-dwh`) y devuelve JSON.
- Las peticiones a cualquier otra ruta reciben el bundle estático de React desde el contenedor frontend.
- Ni la API, ni el frontend, ni las bases de datos exponen puertos directamente al host en la configuración de producción con proxy.

Conclusiones

Al término del proyecto, el sistema planteado en la introducción ha sido implementado en su totalidad. Este capítulo recoge el balance del trabajo realizado, las lecciones aprendidas durante su desarrollo y las posibles líneas de evolución futura.

9.1 Cumplimiento de objetivos

El objetivo principal del proyecto, consistente en crear un almacén de datos que integrara información procedente de múltiples fuentes institucionales y exponerla mediante un cuadro de mando accionable para los gestores públicos, ha sido alcanzado en su totalidad. A continuación se evalúa el grado de consecución de cada uno de los objetivos específicos enunciados en la introducción.

El primer objetivo, la **simulación de fuentes de datos**, se materializó en tres bases de datos PostgreSQL que replican fielmente los dominios de un sistema educativo autonómico real: expedientes académicos, adaptaciones curriculares y encuestas socioeconómicas familiares. La generación de datos sintéticos con distribuciones estadísticas realistas permitió validar el sistema en condiciones representativas sin depender de datos sensibles reales.

El segundo objetivo, el **diseño del almacén de datos**, se resolvió mediante un modelo en estrella con dos tablas de hechos (*fact_rendimiento_anual* y *fact_calificaciones*) y siete dimensiones, siguiendo la metodología de Ralph Kimball. Este diseño permite ejecutar consultas analíticas multidimensionales de forma eficiente y constituye la base sobre la que se calculan todos los KPIs del sistema.

El tercer objetivo, el **diseño de los procesos ETL**, se implementó mediante diez pipelines y tres workflows de orquestación en Apache Hop. El proceso más relevante desde el punto de vista de la lógica de negocio es el cálculo del índice de riesgo de abandono, que integra factores académicos, socioeconómicos y de necesidades educativas especiales en una puntuación normalizada entre 0 y 10, con ventanas de tiempo que permiten comparativas interanuales.

El cuarto objetivo, el **desarrollo del backend**, se concretó en una API RESTful con FastAPI que expone veinte endpoints organizados en cuatro dominios funcionales (autenticación, filtros, vista Macro y vista Micro), protegidos mediante autenticación JWT y desplegados en un contenedor Docker junto al resto del ecosistema.

El quinto y último objetivo, la **creación del cuadro de mando**, se cumplió con la implementación de una aplicación React que ofrece dos vistas complementarias. La vista Macro permite a los técnicos de la administración identificar qué centros concentran mayor riesgo a escala autonómica. La vista Micro actúa como herramienta de diagnóstico causal, desagregando los factores que explican el índice de riesgo de los centros seleccionados. El mecanismo de *drilldown* entre ambas vistas, articulado mediante un contexto de filtros globales, constituye el flujo de trabajo central de la herramienta y responde directamente a la necesidad de fundamentar con evidencia objetiva las decisiones de asignación de recursos.

9.2 Lecciones aprendidas

El desarrollo de este proyecto ha proporcionado una experiencia práctica integral en el ciclo de vida completo de un sistema de Inteligencia de Negocios, desde la ingesta de datos hasta la visualización analítica, que difícilmente puede adquirirse de forma teórica.

Desde el punto de vista técnico, la principal lección aprendida es la importancia del **diseño dimensional como contrato arquitectónico**. Las decisiones tomadas en la fase de modelado del Datamart condicionan directamente la complejidad de los procesos ETL, la expresividad de las consultas SQL y, en última instancia, la velocidad de respuesta de la API. Un modelo en estrella bien diseñado simplifica radicalmente todas las capas superiores del sistema.

En cuanto a los procesos ETL, la adopción de **Apache Hop** ha demostrado ser una elección acertada para un proyecto de esta naturaleza. Su modelo visual de pipelines facilita la comprensión y el mantenimiento de transformaciones complejas, y su soporte nativo para contenedores Docker lo hace especialmente adecuado para arquitecturas modernas. No obstante, la curva de aprendizaje inicial es pronunciada, y la depuración de errores en pipelines encadenados requiere una estrategia sistemática de validación por etapas.

La arquitectura **desacoplada cliente-servidor** ha demostrado su valor a lo largo del desarrollo iterativo: la posibilidad de modificar los umbrales de alerta en el backend o añadir nuevos endpoints sin afectar al frontend, y viceversa, ha agilizado significativamente el ciclo de refinamiento de la lógica de negocio.

Finalmente, la decisión de utilizar exclusivamente **tecnologías Open Source** ha validado que es posible construir un sistema de BI de nivel profesional sin incurrir en costes de licenciamiento, lo que refuerza la viabilidad de este tipo de soluciones en el contexto de la

administración pública, donde la transparencia, la auditabilidad y la sostenibilidad económica son requisitos fundamentales.

9.3 Líneas de trabajo futuro

El sistema desarrollado constituye una base sólida y extensible sobre la que pueden plantearse diversas líneas de evolución que ampliarían su alcance y su impacto.

La mejora más inmediata y de mayor impacto sería la **integración con fuentes de datos reales**. El sistema ha sido diseñado y validado con datos sintéticos, pero su arquitectura ETL está preparada para conectarse a los sistemas de información reales de una administración educativa autónoma con modificaciones mínimas en la capa de extracción.

Una segunda línea de evolución relevante sería la incorporación de **modelos predictivos de aprendizaje automático**. El índice de riesgo actual se calcula mediante reglas heurísticas ponderadas. La sustitución o complementación de este cálculo por un modelo supervisado (por ejemplo, una regresión logística o un árbol de decisión entrenado sobre datos históricos reales) permitiría mejorar la precisión de las alertas tempranas y reducir los falsos positivos.

Desde el punto de vista de la usabilidad, sería valioso incorporar un **módulo de exportación de informes** que permitiera a los técnicos generar documentos PDF o Excel a partir de las vistas activas, facilitando la elaboración de informes institucionales sin necesidad de herramientas externas.

Por último, la **automatización del proceso ETL** mediante un orquestador de tareas (como Apache Airflow) permitiría que el Datamart se actualizara de forma periódica y desatendida al inicio de cada curso académico, eliminando la necesidad de ejecución manual y convirtiendo el sistema en una solución verdaderamente operacional.

Apéndices

Material adicional

EXEMPLO de capítulo con formato de apéndice, onde se pode incluír material adicional que non teña cabida no corpo principal do documento, suxeito á limitación de 80 páxinas establecida no regulamento de TFGs.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque.

Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Bibliografía

- [1] Organización para la Cooperación y el Desarrollo Económicos (OCDE), “Education at a glance: Oecd indicators,” 2023, consultado el 25 de junio de 2026. [En línea]. Disponible en: <https://www.oecd.org/education/education-at-a-glance/>
- [2] Instituto Nacional de Estadística (INE), “Indicadores de pobreza y condiciones de vida,” 2023, consultado el 25 de junio de 2026. [En línea]. Disponible en: <https://www.ine.es/daco/daco42/sociales/pobreza.pdf>
- [3] Ayuda en Acción, “Pobreza absoluta y pobreza relativa: diferencias,” 2023, consultado el 25 de junio de 2026. [En línea]. Disponible en: <https://ayudaenaccion.org/blog/solidaridad/pobreza-absoluta-pobreza-relativa/>
- [4] BBVA Sostenibilidad, “¿qué es la pobreza multidimensional y cómo se mide?” 2023, consultado el 25 de junio de 2026. [En línea]. Disponible en: <https://www.bbva.com/es/sostenibilidad/que-es-la-pobreza-multidimensional-y-como-se-mide/>
- [5] Observatorio de la Pobreza, “Observatorio de la pobreza y la desigualdad,” 2023, consultado el 25 de junio de 2026. [En línea]. Disponible en: <https://www.observatoriopobreza.org/>
- [6] R. Kimball and M. Ross, *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling*, 3rd ed. John Wiley & Sons, 2013.
- [7] The Apache Software Foundation, “Apache hop (hop orchestration platform) documentation,” 2023, consultado el 25 de junio de 2026. [En línea]. Disponible en: <https://hop.apache.org/manual/latest/>
- [8] Docker Inc., “Docker documentation,” 2023, consultado el 25 de junio de 2026. [En línea]. Disponible en: <https://docs.docker.com/>

- [9] The PostgreSQL Global Development Group, “Postgresql 16.0 documentation,” 2023, consultado el 25 de junio de 2026. [En línea]. Disponible en: <https://www.postgresql.org/docs/16/index.html>
- [10] S. Ramírez, “Fastapi documentation,” 2023, consultado el 25 de junio de 2026. [En línea]. Disponible en: <https://fastapi.tiangolo.com/>
- [11] Meta Platforms, Inc., “React: The library for web and native user interfaces,” 2023, consultado el 25 de junio de 2026. [En línea]. Disponible en: <https://react.dev/>
- [12] OECD, “Pisa 2018 technical report,” 2020, consultado el 25 de junio de 2026. [En línea]. Disponible en: <https://www.oecd.org/pisa/data/pisa2018technicalreport/>
- [13] Instituto Nacional de Estadística (INE), “Encuesta de condiciones de vida (ecv),” 2023, consultado el 25 de junio de 2026. [En línea]. Disponible en: https://www.ine.es/dyngs/INEbase/es/operacion.htm?c=Estadistica_C&cid=1254736176807&menu=resultados&idp=1254735976608
- [14] Red Europea de Lucha contra la Pobreza y la Exclusión Social en el Estado Español (EAPN-ES), “Guía para comprender la pobreza,” 2023, consultado el 25 de junio de 2026. [En línea]. Disponible en: <https://eapn-clm.org/guia-para-comprender-la-pobreza/>